Milestone 3 ��� Uber Ride Hailing Platform

Milestone 3 — Uber Ride Hailing Platform

True Microservices: Service Isolation, Inter-Service Communication & Kubernetes

Weight	40% of final grade
Theme	Ride Hailing (Uber)
Deadline	Saturday 17/05/2026 at 11:59 PM

Services in This Theme

Service	Module name	Internal port	Database (M3)
User Service	`user-service`	8080	`uberdb-users`
Driver Service	`driver-service`	8080	`uberdb-drivers`
Ride Service	`ride-service`	8080	`uberdb-rides`
Location Service	`location-service`	8080	`uberdb-locations`
Payment Service	`payment-service`	8080	`uberdb-payments`
API Gateway	`api-gateway`	8080	—

What M3 Adds to Your Codebase

M1 built 5 services sharing one PostgreSQL database. M2 added 6 databases (polyglot persistence), authentication, caching, and design patterns — still one PostgreSQL, still cross-service SQL JOINs inside that PostgreSQL.

M3 finishes the transformation:

- **Database isolation** — each service gets its own PostgreSQL instance. No service can open a JDBC connection to another service's database.
- **OpenFeign** — synchronous HTTP calls replace cross-service SQL JOINs for read dependencies.
- **RabbitMQ** — asynchronous events replace cross-service write side-effects.
- **Spring Cloud Gateway** — a 6th Maven module acts as the single entry point. JWT validation moves here.
- **Kubernetes** — all services and databases deploy to a local MiniKube cluster.

What Does NOT Change

- All 45 M1 features — except the cross-service SQL inside 17 of them (see sections below)
- All 7 M2 design patterns
- All 6 M2 databases (PostgreSQL + MongoDB + Redis + Elasticsearch + Neo4j + Cassandra)
- JWT authentication (shared secret, stays the same)
- Redis caching (all cached endpoints remain cached)
- MongoDB event logging (Observer pattern stays in place)

New Ride Status Values

M3 adds saga-related statuses to the Ride entity's status enum:

New status	When it is set
`PAYMENT_PENDING`	S3 sets this when `payment.initiated` event is consumed
`PAID`	S3 sets this when `payment.completed` event is consumed
`PAYMENT_FAILED`	S3 sets this when `payment.failed` event is consumed
`REFUNDED`	S3 sets this when `payment.refunded` event is consumed

The existing M1 value `COMPLETED` is reused as the saga "ride finished, awaiting payment" status — S3-F4 sets `COMPLETED` immediately before publishing `ride.completed`. Existing `REQUESTED`, `ACCEPTED`, `IN_PROGRESS`, and `CANCELLED` values from M1 remain unchanged.

Section 1 — Database Isolation

1.1 What Changes

Every service previously connected to a single shared database:

```
spring:
  datasource:
    url: jdbc:postgresql://postgres:5432/uberdb
```

In M3, each service connects to its own database on its own PostgreSQL instance:

```
# user-service application.yml
spring:
  datasource:
    url: jdbc:postgresql://user-postgres:5432/uberdb-users

# driver-service application.yml
spring:
  datasource:
    url: jdbc:postgresql://driver-postgres:5432/uberdb-drivers

# ride-service application.yml
spring:
  datasource:
    url: jdbc:postgresql://ride-postgres:5432/uberdb-rides

# location-service application.yml
spring:
  datasource:
    url: jdbc:postgresql://location-postgres:5432/uberdb-locations

# payment-service application.yml
spring:
  datasource:
    url: jdbc:postgresql://payment-postgres:5432/uberdb-payments
```

1.2 Cross-Service FK Columns Become Plain Longs

Every `@ManyToOne` or `@JoinColumn` that pointed to another service's entity becomes a plain `Long` field. The column still exists in the database, but there is no JPA foreign-key relationship across databases. M1 already stored `userId`, `driverId`, and `rideId` as plain `Long` columns (per §7 of the M1 spec) — there is nothing to convert there. The few remaining cross-service relationships that may have crept in via M2 retrofits must be flattened.

Table	Column	Before (M1/M2)	After (M3)
`rides`	`user_id`	Long (already plain)	`private Long userId;`
`rides`	`driver_id`	Long (already plain)	`private Long driverId;`
`locations`	`driver_id`	Long (already plain)	`private Long driverId;`
`payments`	`ride_id`	Long (already plain)	`private Long rideId;`
`payments`	`user_id`	Long (already plain)	`private Long userId;`
`driver_documents`	`driver_id`	`@ManyToOne Driver driver`	unchanged (same service)
`saved_addresses`	`user_id`	`@ManyToOne User user`	unchanged (same service)
`ride_stops`	`ride_id`	`@ManyToOne Ride ride`	unchanged (same service)

The intra-service `@ManyToOne` relationships (SavedAddress→User, DriverDocument→Driver, RideStop→Ride, PaymentCoupon→Payment, PaymentCoupon→Coupon) all stay as JPA-managed because both sides live in the same service's database.

1.3 NoSQL Databases — Shared Instance, Separate Ownership

MongoDB, Redis, Elasticsearch, Neo4j, and Cassandra remain as single shared instances (one StatefulSet each in Kubernetes). Each service already owns its own collections/indexes/keyspace and never reads another service's data — the logical isolation from M2 is sufficient. Running 5 MongoDB + 5 Redis + 5 Elasticsearch + 5 Neo4j + 5 Cassandra StatefulSets would make MiniKube unrunnable.

The M3 rule: No service connects to another service's PostgreSQL. Each service continues to own its MongoDB collections (`auth_events`, `driver_events`, `ride_events`, `location_events`, `payment_audit_trail`), Redis key prefix (`user-service::*`, `driver-service::*`, etc.), Elasticsearch index (`drivers` — driver-service), Neo4j label set (`UserNode`, `DriverNode`, `RODE_WITH` — ride-service), and Cassandra keyspace (`location_tracking_events` — location-service).

1.4 Deliverables for DB Isolation

- `user-service/application.yml` — datasource URL points to `user-postgres:5432/uberdb-users`
- `driver-service/application.yml` — datasource URL points to `driver-postgres:5432/uberdb-drivers`
- `ride-service/application.yml` — datasource URL points to `ride-postgres:5432/uberdb-rides`
- `location-service/application.yml` — datasource URL points to `location-postgres:5432/uberdb-locations`
- `payment-service/application.yml` — datasource URL points to `payment-postgres:5432/uberdb-payments`
- All cross-service `@ManyToOne` fields replaced with `Long` in the relevant entities
- New saga statuses added to Ride status enum

Section 2 — Inter-Service Communication Setup

2.1 OpenFeign Dependency

Add to every service that makes Feign calls:

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

Add Spring Cloud BOM to `dependencyManagement`:

```xml
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>2025.1.1</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

Enable on `@SpringBootApplication`:

```java
@SpringBootApplication
@EnableFeignClients
public class UserServiceApplication { }
```

2.2 Feign Client Pattern (Example)

```java
@FeignClient(name = "ride-service", url = "${feign.ride-service.url}")
public interface RideServiceClient {

    @GetMapping("/api/rides/user/{userId}/summary")
    RideSummaryDTO getUserRideSummary(@PathVariable Long userId);

    @GetMapping("/api/rides/user/{userId}/active-count")
    int getActiveRideCount(@PathVariable Long userId);

    @GetMapping("/api/rides/user/{userId}/completed-count")
    long getCompletedRideCount(@PathVariable Long userId);
}
```

In `application.yml`, add each service to the one that requires it:

```yaml
feign:
  user-service:
    url: http://user-service:8080
  driver-service:
    url: http://driver-service:8080
  ride-service:
    url: http://ride-service:8080
  location-service:
    url: http://location-service:8080
  payment-service:
    url: http://payment-service:8080
```

2.3 Correlation ID Propagation

Every service must forward `X-Correlation-ID` on all outgoing Feign calls:

```
@Configuration
public class FeignCorrelationConfig {

    @Bean
    public RequestInterceptor correlationIdInterceptor() {
        return template -> {
            String correlationId = MDC.get("correlationId");
            if (correlationId != null) {
                template.header("X-Correlation-ID", correlationId);
            }
        };
    }
}
```

2.4 Error Handling

Wrap every Feign call in try-catch. Never let a downstream failure crash the calling service, for example:

```
try {
    RideSummaryDTO summary = rideServiceClient.getUserRideSummary(userId);
    return buildDTO(user, summary);
} catch (FeignException.NotFound e) {
    return buildDTO(user, RideSummaryDTO.empty());
} catch (FeignException e) {
    log.warn("ride-service unavailable for user {}: {}", userId, e.getMessage());
    throw new ServiceUnavailableException("Ride service temporarily unavailable");
}
```

2.5 RabbitMQ Dependency

Add to every service that publishes or consumes events:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

2.6 RabbitMQ Connection Configuration

Add to every service's `application.yml`:

```
spring:
  rabbitmq:
    host: rabbitmq
    port: 5672
    username: guest
    password: guest
    listener:
      simple:
        acknowledge-mode: auto
        default-requeue-rejected: false
        retry:
          enabled: true
          initial-interval: 1000
          max-attempts: 3
```

2.7 Topology — What Each Service Must Declare

Each service declares its own RabbitMQ topology as Spring `@Bean`s in a `@Configuration` class. The responsibilities are split:

- **Producer service** declares only the `TopicExchange` it publishes to.
- **Consumer service** declares the `Queue`, the DLQ, another `TopicExchange` reference (same name — Spring deduplicates), and the `Binding` that connects them.

Every consumer queue must have a dead-letter queue. The DLQ is wired by declaring the consumer queue with the `x-dead-letter-exchange` and `x-dead-letter-routing-key` arguments pointing at a separate dead-letter exchange + DLQ. Combined with `default-requeue-rejected: false` (§2.6), this means: when a listener method throws and Spring's retry exhausts (`max-attempts: 3`), the message is automatically routed to the DLQ — no manual `basicAck`/`basicNack` code in the consumer.

The exchange type for all Uber events is `TopicExchange`. This allows routing key wildcards so future events can be added without declaring a new exchange.

2.8 Event Payload Records

Event payloads are plain Java `record` classes, serialized to JSON by Jackson. They live in an `events` package inside the shared contracts module:

```
// ride-service
public record RidePlacedEvent(Long rideId, Long userId, Long driverId) {}
public record RideCompletedEvent(Long rideId, Long userId, Long driverId, Double fare) {}
public record RideCancelledEvent(Long rideId, Long userId, Long driverId, String reason) {}

// driver-service
public record DriverStatusChangedEvent(Long driverId, String oldStatus, String newStatus) {}
public record DriverRatedEvent(Long driverId, Long rideId, Double rating, Long userId) {}
public record DriverDocumentVerifiedEvent(Long driverId, Long documentId, Long verifiedBy) {}

// payment-service
public record PaymentInitiatedEvent(Long paymentId, Long rideId, Double amount) {}
public record PaymentCompletedEvent(Long paymentId, Long rideId, Double amount) {}
public record PaymentFailedEvent(Long paymentId, Long rideId, String reason) {}
public record PaymentRefundedEvent(Long paymentId, Long rideId, Double refundAmount) {}

// user-service
public record UserRegisteredEvent(Long userId, String email, String role) {}
public record UserDeactivatedEvent(Long userId) {}

// location-service
public record LocationTrackedEvent(Long driverId, Long rideId, Double latitude, Double longitude) {}
```

2.9 Full Event Map (Uber)

Producer	Exchange	Routing key	Payload record	Consumers
user-service	`user.events`	`user.registered`	`UserRegisteredEvent`	ride-service
user-service	`user.events`	`user.deactivated`	`UserDeactivatedEvent`	ride-service
driver-service	`driver.events`	`driver.status-changed`	`DriverStatusChangedEvent`	(observability only)
driver-service	`driver.events`	`driver.rated`	`DriverRatedEvent`	(observability only)
driver-service	`driver.events`	`driver.document.verified`	`DriverDocumentVerifiedEvent`	(audit only)
ride-service	`ride.events`	`ride.placed`	`RidePlacedEvent`	driver-service
ride-service	`ride.events`	`ride.completed`	`RideCompletedEvent`	user, driver, location, payment
ride-service	`ride.events`	`ride.cancelled`	`RideCancelledEvent`	user, driver, location, payment
location-service	`location.events`	`location.tracked`	`LocationTrackedEvent`	(audit only)
payment-service	`payment.events`	`payment.initiated`	`PaymentInitiatedEvent`	ride-service
payment-service	`payment.events`	`payment.completed`	`PaymentCompletedEvent`	ride-service
payment-service	`payment.events`	`payment.failed`	`PaymentFailedEvent`	ride-service
payment-service	`payment.events`	`payment.refunded`	`PaymentRefundedEvent`	ride-service

2.10 Authorization Convention for Path-Param Endpoints

Every endpoint that takes a `{userId}` or `{driverId}` path-param must verify ownership before returning data. The api-gateway propagates the JWT-authenticated caller as `X-User-Id` and `X-User-Role` headers (see §9.3). The convention is:

Ownership rule: The path-param `{userId}` or `{driverId}` must equal the caller's `X-User-Id`, OR the caller's `X-User-Role` must equal `ADMIN`. Otherwise → throw 403 Forbidden ("not authorized to access this resource").

Caller-existence rule: Before any business logic runs, look up the caller (via local query or Feign to user-service for cross-service paths) — if not found, → throw 404 Not Found ("caller user not found"). This catches the case where a JWT is valid but the underlying user has been deleted/deactivated.

Endpoints affected by this convention (non-exhaustive — apply universally):

Endpoint	Service	Ownership check
`GET /api/users/{id}/ride-summary` (S1-F3)	S1	`id == X-User-Id` OR caller is ADMIN
`PUT /api/users/{id}/deactivate` (S1-F4)	S1	`id == X-User-Id` OR caller is ADMIN
`GET /api/drivers/{id}/earnings` (S2-F3)	S2	The driver `{id}` must equal the caller's driver-user-id, OR caller is ADMIN
`GET /api/drivers/{id}/dashboard` (S2-F12)	S2	Same as above
`GET /api/rides/recommendations?userId=` (S3-F12)	S3	`userId == X-User-Id` OR caller is ADMIN
`GET /api/payments/user/{id}/summary` (S5-F3)	S5	`id == X-User-Id` OR caller is ADMIN

The per-feature Behavior sections in §3–§7 reference this rule by writing "Auth: ownership rule (§2.10)" instead of repeating the full check on every feature. Aggregate report endpoints (e.g., S1-F6 top-riders, S2-F6 top-rated-drivers, S5-F1 revenue reports) require role = ADMIN — those declare "Auth: ADMIN only" in their per-feature blocks.

2.11 Publish-After-Commit Semantics (No Outbox)

Every publisher in this spec follows the same rule: **commit the local PostgreSQL transaction first, then publish to RabbitMQ.** This is a deliberate trade-off — M3 does not require a transactional outbox pattern. The implication:

> *If the local commit succeeds but the RabbitMQ publish fails (network blip, broker outage, OOM between the two operations), downstream consumers will never see the event.*

This is acceptable for M3 academic scope because:

1. **All critical events flow through DLQ + retry.** RabbitMQ consumers configured per §2.6 (`acknowledge-mode: auto`, `default-requeue-rejected: false`, `max-attempts: 3`) bounce failed deliveries to a DLQ. A separate operator-driven re-drive can replay the DLQ. The producer's lost-event window is the only gap.
2. **The saga's compensating reads close the gap for the highest-stakes flows.** After a `payment-completed` publish, the rider polls `GET /api/rides/{id}` to see status=PAID — if the ride is still PAYMENT_PENDING after a reasonable wait, the rider can re-trigger payment (S5-F4 is naturally state-machine-idempotent on PENDING).
3. **Consumer handlers must be idempotent.** Because retries are guaranteed under at-least-once delivery, every consumer handler must check current state before mutating (e.g., S2's `ride.placed` consumer no-ops when the driver is already BUSY; S3's `payment.completed` consumer checks ride status before flipping to PAID). Idempotency is the prerequisite that makes at-least-once safe.

For real-world systems, the recommended hardening is the **transactional outbox pattern** — write the event to a local outbox table inside the same PG transaction as the business mutation, then a separate poller drains the outbox to RabbitMQ. This guarantees at-least-once publish even across crashes. It is a §15 Bonus item, not part of the M3 baseline.

2.12 N+1 Feign Hazards (Candidate-Set Caps)

Several features iterate over a candidate set and make a Feign call per element. This is N+1 by construction. Examples in this spec: S1-F6 (per-user payment totals), S1-F9 (per-user completed-count), S3-F12 (per-driver enrichment for recommendations), S4-F3 (per-driver status filter for nearby search), S4-F9 (per-driver name enrichment for stationary), S5-F10 (per-driver vehicleType for surge-fee categorization).

Cap rule: Every M3 feature that does per-element Feign calls must cap the candidate set to **at most 100 elements** at the local-DB query stage (`LIMIT 100`). Beyond 100, return what fits and document the truncation in the response (or in a header). Larger result sets need a batch-endpoint optimization — see §15 Bonus.

The cap protects the downstream service from accidental fan-out (e.g., a top-riders query against a 10K-user table making 10K Feign calls). For students who hit the cap during testing, the recommended optimization is to expose a batch endpoint on the called service (e.g., `POST /api/payments/users/totals` accepting a list of userIds and returning a list of totals) — but this is a Bonus, not a baseline requirement. Each per-feature Behavior in §3–§7 with this hazard says “Cap candidate set at 100 (§2.12)” inline.

Section 3 — User Service Refactoring (S1)

New Endpoints S1 Must Expose

No new external endpoints — user-service already exposes `GET /api/users/{id}` (M1 CRUD). Downstream services call this existing endpoint.

Features That Require Feign Calls

[S1-F3] Get User Ride Summary

- **Branch:** `feat/M3/user/S1-F3/<studentID>`
- **Endpoint:** `GET /api/users/{id}/ride-summary`
- **Auth:** ownership rule (§2.10) — `{id}` must equal `X-User-Id` or caller must be ADMIN; caller-existence rule (§2.10) — 404 if `{id}` not in user-postgres.

M1 implementation: Direct native SQL JOIN on the shared `rides` table using `user_id` to compute total/completed/cancelled counts and total spent.

M3 change: Replace the SQL JOIN with a Feign call to ride-service, so that the interface would be:

```
@FeignClient(name = "ride-service", url = "${feign.ride-service.url}")
public interface RideServiceClient {
    @GetMapping("/api/rides/user/{userId}/summary")
    RideSummaryDTO getUserRideSummary(@PathVariable Long userId);
}
```

`RideSummaryDTO` returned by ride-service: `{totalRides, completedRides, cancelledRides, totalSpent, averageFare}`

user-service calls this and merges it with the local User data to build `UserRideSummaryDTO`.

Test scenario:

1. *(setup)* In user-postgres: create User ID=1. In ride-postgres: create 5 rides for userId=1 — 3 COMPLETED (fares 50, 75, 100), 1 CANCELLED, 1 REQUESTED.
2. *(action)* `GET /api/users/1/ride-summary` with valid Bearer token.
3. *(expect)* 200 — totalRides=5, completedRides=3, cancelledRides=1, totalSpent=225.00, averageFare=75.00.
4. *(verify)* No direct JDBC connection from user-postgres to ride-postgres. The ride counts come from a Feign HTTP call.

[S1-F4] Deactivate User Account

- **Branch:** `feat/M3/user/S1-F4/<studentID>`
- **Endpoint:** `PUT /api/users/{id}/deactivate`
- **Auth:** ownership rule (§2.10) — `{id}` must equal `X-User-Id` or caller must be ADMIN; caller-existence rule (§2.10) — 404 if `{id}` not in user-postgres. Re-deactivating an already-DEACTIVATED user is idempotent → return 200 (no event re-published).

M1 implementation: `SELECT COUNT(*) FROM rides WHERE user_id = ? AND status IN ('REQUESTED','ACCEPTED','IN_PROGRESS')` directly on the shared database.

M3 change: Replace with a Feign call to ride-service `GET /api/rides/user/{userId}/active-count` → returns `int`.

If the returned count > 0, throw 400 (“User has active rides”). Otherwise set status = DEACTIVATED and save.

After deactivation: publish `user.deactivated` RabbitMQ event (see §2.9).

Test scenario:

1. *(setup)* User ID=1 in user-postgres. Ride in ride-postgres: userId=1, status=REQUESTED.
2. *(action)* `PUT /api/users/1/deactivate` → Feign → ride-service returns active-count=1.
3. *(expect)* 400 — cannot deactivate user with active rides.
4. *(setup)* Update the ride status to COMPLETED in ride-postgres.
5. *(action)* `PUT /api/users/1/deactivate` → Feign returns active-count=0.
6. *(expect)* 200 — user status = DEACTIVATED in user-postgres. RabbitMQ `user.deactivated` event published.

[S1-F6] Top Riders by Spending

- **Branch:** `feat/M3/user/S1-F6/<studentID>`
- **Endpoint:** `GET /api/users/reports/top-riders?startDate={date}&endDate={date}&limit={n}`

M1 implementation: `JOIN users ON payments.user_id = users.id GROUP BY users.id ORDER BY SUM(amount) DESC` against the shared `payments` table.

M3 change: user-service cannot JOIN the `payments` table (it lives in payment-postgres). Instead:

1. Fetch all users from user-postgres.
2. For each user, call Feign → `GET /api/payments/user/{userId}/total` on payment-service → returns `BigDecimal` (total COMPLETED payment amount for this user in the date range).
3. Sort users by total descending, take first `limit`.
4. Build and return `List<TopRiderDTO>` where `TopRiderDTO = {userId, name, totalSpent}` (no rideCount — that would require a second Feign call to ride-service per user; if you want it, see the Bonus item in §15).

Candidate-set cap (N+1 hazard): This pattern is N+1 by construction (one Feign call per candidate user). For M3, cap the local-DB candidate query to at most 100 users — e.g., `LIMIT 100` on the user-postgres SELECT. Larger datasets need a batch-endpoint optimization (§15 Bonus).

Note on date filtering: Pass `startDate` and `endDate` as query params to the payment-service endpoint so it filters server-side rather than fetching all payments, so that the interface would be:

```
@FeignClient(name = "payment-service", url = "${feign.payment-service.url}")
public interface PaymentServiceClient {
    @GetMapping("/api/payments/user/{userId}/total")
    BigDecimal getUserPaymentTotal(
        @PathVariable Long userId,
        @RequestParam String startDate,
        @RequestParam String endDate
    );
}
```

Test scenario:

1. *(setup)* 3 users in user-postgres. In payment-postgres: User A = 300 total, User B = 500 total, User C = 100 total, all within March 2026.
2. *(action)* `GET /api/users/reports/top-riders?startDate=2026-03-01&endDate=2026-03-31&limit=2`.
3. *(expect)* 200 — `[{userId: B, name: "...", totalSpent: 500.00}, {userId: A, name: "...", totalSpent: 300.00}]`. User C excluded (totalSpent=100 falls below the top-2 cutoff).
4. *(verify)* user-service made Feign calls to payment-service for each user's total. No direct query on `payments` table.

[S1-F9] Find Users by Language Preference with Minimum Rides

- **Branch:** `feat/M3/user/S1-F9/<studentID>`
- **Endpoint:** `GET /api/users/preferences/language?lang={lang}&minRides={n}`

M1 implementation: JSONB query on `users` table + subquery counting `rides` rows by `user_id` with status=COMPLETED.

M3 change:

1. Query user-postgres for users whose `preferences->>'language'` matches the given value (cap at 100 —

see §2.12).

2. For each matching user, call Feign → `GET /api/rides/user/{userId}/completed-count` → returns `long`.
3. Keep only users whose returned count ≥ minRides.

Test scenario:

1. *(setup)* 3 users in user-postgres: User A and User B have language=ar, User C has language=en. In ride-postgres: User A has 5 COMPLETED rides, User B has 2 COMPLETED rides, User C has 10 COMPLETED rides.
2. *(action)* `GET /api/users/preferences/language?lang=ar&minRides=3`.
3. *(expect)* 200 — only User A returned (User B has 2 completed rides, below threshold).
4. *(verify)* Feign call made to ride-service for each candidate user.

RabbitMQ: S1 Publishes

Routing key	Exchange	Payload	When
`user.registered`	`user.events`	`{userId, email, role}`	After successful user registration (S1-F10)
`user.deactivated`	`user.events`	`{userId}`	After S1-F4 successfully sets DEACTIVATED

RabbitMQ: S1 Consumes

Routing key	From exchange	Action
`ride.completed`	`ride.events`	Update user's ride stats (increment total rides, total spent) in user-postgres
`ride.cancelled`	`ride.events`	Reverse user's ride stats (decrement total rides, subtract amount) in user-postgres

Queue declaration: `user.ride.saga-listener` with DLQ `user.ride.saga-listener.dlq`.

S1 Deliverables

- Remove any cross-service `@ManyToOne` on User / SavedAddress entities (M1 already kept these intra-service)
- `feign.ride-service.url` and `feign.payment-service.url` in application.yml
- `RideServiceClient` Feign interface with `getUserRideSummary`, `getActiveRideCount`, `getCompletedRideCount`
- `PaymentServiceClient` Feign interface with `getUserPaymentTotal`
- S1-F3 refactored to use Feign → ride-service
- S1-F4 refactored to use Feign → ride-service; publishes `user.deactivated` after success
- S1-F6 refactored to use Feign → payment-service per user
- S1-F9 refactored to use Feign → ride-service per matching user
- RabbitMQ `user.events` TopicExchange declared
- `user.registered` published on registration
- `user.deactivated` published on S1-F4
- Consumer for `ride.completed` and `ride.cancelled` with auto ACK + DLQ via `x-dead-letter-exchange`
- `logback-spring.xml` with Loki4J appender (see §11)

Section 4 — Driver Service Refactoring (S2)

New Endpoints S2 Must Expose

These endpoints are called by other services via Feign. They must exist before S1, S3, S4, S5 SYNC branches are merged.

Endpoint	Called by	Returns	Description
`GET /api/drivers/{id}`	S3, S4, S5	`DriverDTO`	Already exists (M1 CRUD). Verify it returns id, name, phone, status, rating, totalRatings, vehicleDetails.
`GET /api/drivers/{id}/availability`	S3 (S3-F2 pre-check)	`{"status": String}`	Convenience endpoint returning the driver's current status value. 404 if driver not found.

[S2-F3] Get Driver Earnings Summary

- **Branch:** `feat/M3/driver/S2-F3/<studentID>`
- **Endpoint:** `GET /api/drivers/{id}/earnings?startDate={date}&endDate={date}`
- **Auth:** ownership rule (§2.10) — driver `{id}` must equal the caller's driver-user-id, OR caller must be ADMIN; driver-existence rule — 404 if `{id}` not in driver-postgres.

M1 implementation: Native SQL aggregation on the shared `rides` table for completed rides assigned to this driver.

M3 change: Replace with Feign → ride-service.

```
@FeignClient(name = "ride-service", url = "${feign.ride-service.url}")
public interface RideServiceClient {
    @GetMapping("/api/rides/driver/{driverId}/summary")
    DriverRideSummaryDTO getDriverRideSummary(
        @PathVariable Long driverId,
        @RequestParam String startDate,
        @RequestParam String endDate
    );
}
```

`DriverRideSummaryDTO` from ride-service: `{totalRides, totalEarnings, averageFare}`

driver-service merges this with the local Driver entity to build `DriverEarningsDTO`.

Test scenario:

1. *(setup)* Driver ID=1 in driver-postgres. 5 COMPLETED rides in ride-postgres for driverId=1 in March 2026 with fares 50, 60, 70, 80, 90.
2. *(action)* `GET /api/drivers/1/earnings?startDate=2026-03-01&endDate=2026-03-31`.
3. *(expect)* 200 — totalRides=5, totalEarnings=350.00, averageFare=70.00.
4. *(verify)* No direct JOIN on ride-postgres from driver-service.

[S2-F4] Update Driver Availability

- **Branch:** `feat/M3/driver/S2-F4/<studentID>`
- **Endpoint:** `PUT /api/drivers/{id}/availability`
- **Request body:** `{"status": "OFFLINE" | "AVAILABLE" | "BUSY"}`

M1 implementation: When transitioning to OFFLINE, runs `SELECT COUNT(*) FROM rides WHERE driver_id = ? AND status IN ('ACCEPTED','IN_PROGRESS')` on the shared database to block the change if any active ride exists.

M3 change: Replace the SQL count with Feign → ride-service `GET /api/rides/driver/{driverId}/active-count`. If the count > 0 and the requested status is OFFLINE, throw 400. Otherwise update the local driver row and save.

After the local update, publish `driver.status-changed` to the `driver.events` exchange.

Concurrency trade-off (eventually consistent): Between the Feign read returning active-count=0 and the local UPDATE committing, a `ride.placed` event for this driver may arrive on the S2 consumer (which independently flips the driver to BUSY — see §4 RabbitMQ consumers). In that race, the consumer wins and the driver ends up BUSY despite the OFFLINE intent. This is acceptable for M3 academic scope — the rider whose ride was just assigned still has a driver. The driver can re-issue the OFFLINE request once the ride completes. For real-world systems, harden by either (a) wrapping the read+update in `@Transactional(isolation = REPEATABLE_READ)` and re-fetching active-count inside the transaction, or (b) using a Redis SETNX lock on `driver::lock::{driverId}` while the OFFLINE transition is in flight. Both are out of scope for the M3 baseline; see §15 Bonus.

Test scenario:

1. *(setup)* Driver ID=1 in driver-postgres (status=BUSY). Ride in ride-postgres: driverId=1, status=IN_PROGRESS.
2. *(action)* `PUT /api/drivers/1/availability` body `{"status": "OFFLINE"}` → Feign returns active-count=1.
3. *(expect)* 400 — cannot go OFFLINE with active rides.
4. *(setup)* Update the ride status to COMPLETED in ride-postgres.
5. *(action)* `PUT /api/drivers/1/availability` body `{"status": "OFFLINE"}` → Feign returns active-count=0.
6. *(expect)* 200 — driver status=OFFLINE. `driver.status-changed` event published.

[S2-F7] Rate a Driver After Ride

- **Branch:** `feat/M3/driver/S2-F7/<studentID>`
- **Endpoint:** `POST /api/drivers/{id}/rate`
- **Request body:** `{"rideId": Long, "rating": Double}`

M1 implementation: `SELECT * FROM rides WHERE id = ? AND driver_id = ? AND status = 'COMPLETED'` on the shared database.

M3 change: Replace with Feign → ride-service `GET /api/rides/{rideId}` (reuses existing M1 CRUD endpoint). Validate the returned ride:

- Exists (Feign returns 404 → throw 404)
- `driverId` matches the driver being rated (mismatch → throw 400)
- `status = COMPLETED` (wrong status → throw 400)
- `rating` is between 1 and 5 (invalid → throw 400)

If all checks pass, atomically update the running average via a single SQL statement (do not read-modify-write — concurrent rates of the same driver would lose updates):

```
        UPDATE drivers
        SET rating = (rating * totalRatings + :newRating) / (totalRatings + 1),
            totalRatings = totalRatings + 1
        WHERE id = :driverId;
```

Or the JPQL equivalent:

```
        @Modifying
        @Query("UPDATE Driver d SET d.rating = (d.rating * d.totalRatings + :newRating) / (d.totalRatings + 1),
d.totalRatings = d.totalRatings + 1 WHERE d.id = :driverId")
        int applyRating(@Param("driverId") Long driverId, @Param("newRating") Double newRating);
```

The row-level lock PostgreSQL takes during UPDATE serializes concurrent calls, so two simultaneous rates of the same driver both end up reflected in the running average. After the UPDATE, publish `driver.rated` RabbitMQ event with `{driverId, rideId, rating, userId}`.

Test scenario:

1. *(setup)* Driver ID=1 in driver-postgres (rating=0.0, totalRatings=0). Ride ID=10 in ride-postgres: driverId=1, userId=7, status=COMPLETED.
2. *(action)* `POST /api/drivers/1/rate` body `{"rideId": 10, "rating": 5}`.
3. *(expect)* 200 — driver rating=5.0, totalRatings=1. `driver.rated` event published.
4. *(action)* Same call with a second COMPLETED ride and rating=3 → 200, rating=4.0, totalRatings=2.
5. *(action)* `POST /api/drivers/1/rate` body `{"rideId": 99, "rating": 4.0}` → Feign returns 404 → throw 404.
6. *(action)* `POST /api/drivers/1/rate` body `{"rideId": 10, "rating": 4.0}` for a ride whose driverId is different → 400.
7. *(action)* `POST /api/drivers/1/rate` body `{"rideId": 10, "rating": 6}` → 400 (out of range).

[S2-F8] Verify Driver Document

- **Branch:** `feat/M3/driver/S2-F8/<studentID>`
- **Endpoint:** `PUT /api/drivers/{driverId}/documents/{documentId}/verify`
- **Request body:** *(none — verifier identity is derived from the JWT, not the request body)*

M1 implementation: Local validation on driver + document; then `SELECT role FROM users WHERE id = ?` against the shared `users` table to confirm the verifiedBy user has role ADMIN.

M3 change: The verifier identity is **not taken from the request body** — it is the JWT-authenticated caller, propagated by the api-gateway as the `X-User-Id` header (see §9.3). This closes a privilege-escalation surface: a non-admin caller could otherwise pass any admin's user-id in the body and the spec would happily look that admin up. With JWT-derived identity, the caller can only ever be themselves.

The user lookup uses Feign to user-service:

```
@FeignClient(name = "user-service", url = "${feign.user-service.url}")
public interface UserServiceClient {
    @GetMapping("/api/users/{id}")
    UserDTO getUser(@PathVariable Long id);
}
// In the controller:
@PutMapping("/{driverId}/documents/{documentId}/verify")
public ResponseEntity<Void> verifyDocument(
        @PathVariable Long driverId,
        @PathVariable Long documentId,
        @RequestHeader("X-User-Id") Long callerId) { ... }
```

After the Feign call to `getUser(callerId)`:

- 404 from Feign → throw 403 ("verifier user not found")
- role ≠ ADMIN → throw 403 ("verifier is not an admin")

If admin verification passes and the document is unexpired and belongs to the driver, set `verified = true`, update the document's JSONB metadata with `verifiedAt` and `verifiedBy = callerId`, save. Publish `driver.document.verified` to `driver.events`.

Test scenario:

1. *(setup)* Driver ID=1 with DriverDocument ID=10 (expiryDate=2027-12-31, verified=false). User ID=3 in user-postgres with role=ADMIN.
2. *(action)* `PUT /api/drivers/1/documents/10/verify` with header `X-User-Id: 3` (admin user 3's JWT, gateway-injected).
3. *(expect)* 200 — document verified=true. JSONB metadata contains verifiedAt + verifiedBy=3. `driver.document.verified` published.
4. *(action)* Same call with `X-User-Id: 99` (no such user) → Feign 404 → throw 403.
5. *(action)* `X-User-Id: 2` where user 2's role=RIDER → throw 403.
6. *(action)* Document with expiryDate in the past → 400 (no Feign call needed; local validation fails first).

[S2-F12] Get Driver Performance Dashboard (M2)

- **Branch:** `feat/M3/driver/S2-F12/<studentID>`
- **Endpoint:** `GET /api/drivers/{id}/dashboard`

M2 implementation: Aggregates `totalRides`, `totalEarnings`, `averageRideFare` by joining `rides JOIN payments ON payments.ride_id = rides.id WHERE rides.driver_id = ?` on the shared database, then reads `rating` and `totalRatings` from the local driver row.

M3 change: The cross-service aggregation becomes a Feign call to ride-service — reuse the same `GET /api/rides/driver/{driverId}/summary` endpoint from S2-F3. The driver's own `rating` and `totalRatings` are still read from the local `drivers` table.

Test scenario:

1. *(setup)* Driver ID=5 in driver-postgres (rating=4.5, totalRatings=100). In ride-postgres: 5 COMPLETED rides for driverId=5 with fares 100, 200, 150, 300, 250.
2. *(action)* `GET /api/drivers/5/dashboard` with valid Bearer token.
3. *(expect)* 200 — totalRides=5, totalEarnings=1000, averageRideFare=200, averageRating=4.5, totalRatings=100.
4. *(action)* `GET /api/drivers/999/dashboard` → 404.

Features Verified as NOT Cross-Service (Uber-Specific)

- **S2-F1** (Search Drivers by Status and Rating Range) — Reads only the local `drivers` table. No M3 change.
- **S2-F2** (Update Vehicle Details — JSONB partial update) — Reads/writes only the local `drivers` row. No M3 change.
- **S2-F5** (Filter Drivers by Vehicle Type) — Local JSONB query on `drivers`. No M3 change.
- **S2-F6** (Top Rated Drivers Report) — Response DTO is `{driverId, name, rating, totalRides}`. The `totalRides` field comes from the local driver row's `totalRatings` (which the M1 spec uses as a proxy for completed rides count) — re-verify against the M1 implementation: if your team chose to derive `totalRides` from a JOIN on `rides`, refactor it to call `GET /api/rides/driver/{driverId}/completed-count` per candidate. If `totalRides` came from `totalRatings`, no M3 change is needed.
- **S2-F9** (Get Drivers with Expired Documents) — Local query on `drivers JOIN driver_documents` (both intra-service). No M3 change.
- **S2-F10** (Full-Text Driver Search) — Reads Elasticsearch + the local driver row for enrichment. No M3 change.
- **S2-F11** (Index Driver for Search) — Reads the local driver row, writes ES, logs to MongoDB. No M3 change.

RabbitMQ: S2 Publishes

Routing key	Exchange	Payload	When
`driver.status-changed`	`driver.events`	`{driverId, oldStatus, newStatus}`	After S2-F4 updates driver status
`driver.rated`	`driver.events`	`{driverId, rideId, rating, userId}`	After S2-F7 rating submission
`driver.document.verified`	`driver.events`	`{driverId, documentId, verifiedBy}`	After S2-F8 successfully verifies

RabbitMQ: S2 Consumes

Routing key	From exchange	Action
`ride.placed`	`ride.events`	Set the assigned driver's status to BUSY in driver-postgres (replaces the M1 direct SQL inside S3-F2). If the driver is already BUSY (concurrent assignment race), the consumer is a no-op — log a WARN and skip the update; do not throw or DLQ.
`ride.completed`	`ride.events`	Set the assigned driver's status back to AVAILABLE; increment driver statistics (total completed rides) in driver-postgres
`ride.cancelled`	`ride.events`	Set the assigned driver's status back to AVAILABLE in driver-postgres (replaces the M1 native-SQL update inside S3-F7); reverse driver statistics if previously incremented

Queue declaration: `driver.ride.saga-listener` with DLQ `driver.ride.saga-listener.dlq`.

S2 Deliverables

- `GET /api/drivers/{id}/availability` endpoint implemented in driver-service
- `feign.ride-service.url` and `feign.user-service.url` in driver-service application.yml
- `RideServiceClient` Feign interface with `getDriverRideSummary`, `getActiveRideCount`, `getRide`
- `UserServiceClient` Feign interface with `getUser`
- S2-F3 refactored to use Feign → ride-service
- S2-F4 refactored to use Feign → ride-service; publishes `driver.status-changed`
- S2-F7 refactored to use Feign → ride-service for ride validation; publishes `driver.rated`
- S2-F8 refactored to use Feign → user-service for ADMIN verification; publishes `driver.document.verified`
- S2-F12 refactored to use Feign → ride-service for ride/payment aggregation
- RabbitMQ `driver.events` TopicExchange declared
- Consumer for `ride.placed`, `ride.completed`, `ride.cancelled` with auto ACK + DLQ via `x-dead-letter-exchange`
- `logback-spring.xml` with Loki4J appender

Section 5 — Ride Service Refactoring (S3)

New Endpoints S3 Must Expose

These are called by S1, S2, S4, S5 via Feign. All must be implemented before the EVENTS branches merge.

Endpoint	Called by	Returns	Description
`GET /api/rides/user/{userId}/summary`	S1 (S1-F3)	`RideSummaryDTO`	`{totalRides, completedRides, cancelledRides, totalSpent, averageFare}`
`GET /api/rides/user/{userId}/active-count`	S1 (S1-F4)	`int`	Count of rides with status IN (REQUESTED, ACCEPTED, IN_PROGRESS, COMPLETED, PAYMENT_PENDING)
`GET /api/rides/user/{userId}/completed-count`	S1 (S1-F9)	`long`	Count of rides for this user with status IN (COMPLETED, PAID)
`GET /api/rides/driver/{driverId}/summary`	S2 (S2-F3, S2-F12)	`DriverRideSummaryDTO`	`{totalRides, totalEarnings, averageFare}` for completed rides in optional date range
`GET /api/rides/driver/{driverId}/active-count`	S2 (S2-F4)	`int`	Count of rides with status IN (ACCEPTED, IN_PROGRESS, COMPLETED, PAYMENT_PENDING)
`GET /api/rides/driver/{driverId}/completed-count`	S2 (S2-F6)	`long`	Count of completed rides for top-rated report
`GET /api/rides/{rideId}`	S2 (S2-F7), S5 (S5-F4), S4	`RideDTO`	Already exists (M1 CRUD). Verify it returns id, userId, driverId, status, fare, requestedAt, completedAt, metadata.

[S3-F2] Assign Driver to Ride

- **Branch:** `feat/M3/ride/S3-F2/<studentID>`
- **Endpoint:** `PUT /api/rides/{rideId}/assign?driverId={driverId}`

M1 implementation: `SELECT * FROM drivers WHERE id = ? AND status = 'AVAILABLE'` on the shared database, then `UPDATE drivers SET status = 'BUSY' WHERE id = ?`.

M3 change: Replace the read with Feign → driver-service `GET /api/drivers/{id}`. Validate:

- 404 from Feign → throw 404 (“Driver not found”)
- status ≠ AVAILABLE → throw 400 (“Driver is not available”)

If valid: set `ride.driverId = driverId`, set ride status = ACCEPTED, save. **Do not** update the driver’s status directly. Instead, publish `ride.placed` to `ride.events`. The driver-service consumer of `ride.placed` flips the driver’s status to BUSY in its own database (see §4 RabbitMQ consumers).

Concurrency trade-off (eventually consistent): Two riders racing to assign the same driver — both Feign reads return AVAILABLE, both `PUT /assign` succeed, both publish `ride.placed`. The driver-service consumer picks up event A first → driver flips to BUSY. Event B arrives, sees BUSY → consumer logs a WARN and skips (see §4 — *If the driver is already BUSY … the consumer is a no-op*). The result: ride B is in status=ACCEPTED in ride-postgres but the driver doesn’t know about it (orphan acceptance). For M3 scope, the orphan ride is recovered by the rider via S3-F7 cancel (which publishes `ride.cancelled` and the

driver-service consumer is a no-op for that driver since they never saw the ride). For real-world systems, harden via a Redis SETNX lock on `driver::lock::{driverId}` or `SELECT FOR UPDATE` on the driver row before publishing `ride.placed` — both are §15 Bonus items.

```
@FeignClient(name = "driver-service", url = "${feign.driver-service.url}")
public interface DriverServiceClient {
    @GetMapping("/api/drivers/{id}")
    DriverDTO getDriver(@PathVariable Long id);

    @GetMapping("/api/drivers/{id}/availability")
    DriverAvailabilityDTO getDriverAvailability(@PathVariable Long id);
}
```

Test scenario:

1. *(setup)* REQUESTED ride ID=1 in ride-postgres (no driver assigned). Driver ID=10 in driver-postgres, status=AVAILABLE.
2. *(action)* `PUT /api/rides/1/assign?driverId=10` → Feign returns driver with status=AVAILABLE.
3. *(expect)* 200 — ride status=ACCEPTED, driverId=10. `ride.placed` event published. After event processing: driver-postgres has driverId=10, status=BUSY.
4. *(action)* `PUT /api/rides/1/assign?driverId=10` → ride is now ACCEPTED, not REQUESTED → 400.
5. *(action)* Use a driver with status=BUSY → Feign returns BUSY → 400.
6. *(action)* Use driverId=999 → Feign throws 404 → 404.

S3-F4 and S3-F7 are RabbitMQ-based changes, not Feign. They are fully described in **Section 8 (Ride Lifecycle Saga & Cancellation Cascade)**.

[S3-F11] Record User-Driver Riding Pattern (M2)

- **Branch:** `feat/M3/ride/S3-F11/<studentID>`
- **Endpoint:** `POST /api/rides/{rideId}/record-interaction`

M2 implementation: Direct SQL on shared database: `SELECT name FROM users WHERE id = ?` and `SELECT name, vehicleDetails->>'vehicleType' FROM drivers WHERE id = ?` to fetch the user's name and the driver's name + vehicleType for the Neo4j node creation.

M3 change: Replace both SQL queries with Feign calls.

```
// Get user details from user-service
UserDTO user = userServiceClient.getUser(ride.getUserId());

// Get driver details from driver-service
DriverDTO driver = driverServiceClient.getDriver(ride.getDriverId());
String vehicleType = (String) driver.getVehicleDetails().getOrDefault("vehicleType", "");

// Then proceed with Neo4j graph write as in M2
// (UserNode, DriverNode, RODE_WITH relationship with rideCount + lastRideDate + idempotency marker)
```

The rest of the feature (Neo4j idempotency via the `recorded_ride_ids` collection on the relationship, RODE_WITH increment, MongoDB `INTERACTION_RECORDED` event logging) is unchanged from M2.

Test scenario:

1. *(setup)* User ID=1 in user-postgres (name="Ahmed Hassan"). Driver ID=5 in driver-postgres (name="Omar Khaled", vehicleType=SEDAN). Completed ride ID=10 in ride-postgres: userId=1, driverId=5, status=COMPLETED.
2. *(action)* `POST /api/rides/10/record-interaction`.
3. *(expect)* 200. Neo4j has `(UserNode {userId:1, name:"Ahmed Hassan"})-[:RODE_WITH {rideCount:1}]->(DriverNode {driverId:5, name:"Omar Khaled", vehicleType:"SEDAN"})`. MongoDB `ride_events` has an `INTERACTION_RECORDED` document.
4. *(action)* Repeat the same `POST /api/rides/10/record-interaction` → 200, but rideCount stays at 1 (idempotency — the `recorded_ride_ids` set already contains 10).

5. *(verify)* Feign calls made to user-service and driver-service. No direct SQL on user-postgres or driver-postgres from ride-service.

[S3-F12] Get Driver Recommendations for User (M2)

- **Branch:** `feat/M3/ride/S3-F12/<studentID>`
- **Endpoint:** `GET /api/rides/recommendations?userId={id}&limit={n}`
- **Auth:** ownership rule (§2.10) — `userId` must equal `X-User-Id` or caller must be ADMIN; caller-existence rule — Feign call to user-service returns 404 → throw 404. Cap candidate set at 100 driver IDs (§2.10 N+1 guidance).

M2 implementation: After traversing the Neo4j graph for candidate driver IDs, runs `SELECT name, vehicleDetails FROM drivers WHERE id IN (?)` against the shared `drivers` table to enrich the recommendations.

M3 change: Replace the SQL enrichment with Feign calls to driver-service.

```
        // Verify the requesting user exists
        UserDTO user = userServiceClient.getUser(userId);

        // For each candidate driverId from Neo4j graph traversal:
        DriverDTO driver = driverServiceClient.getDriver(candidateId);
        String vehicleType = (String) driver.getVehicleDetails().getOrDefault("vehicleType", "");
        recommendations.add(new DriverRecommendationDTO(driver.getId(), driver.getName(), vehicleType, score));
```

The Neo4j collaborative-filtering traversal logic (find users who rode with the same drivers, then drivers those users rode with that this user has not), the limit, and the cache TTL are unchanged from M2.

Test scenario:

1. *(setup)* Users U1, U2, U3 in user-postgres. Drivers D1, D2, D3, D4 in driver-postgres (D4 vehicleType=LUXURY). Neo4j has interactions: U1→D1, U1→D2; U2→D1, U2→D3; U3→D2, U3→D4.
2. *(action)* `GET /api/rides/recommendations?userId={U1.id}&limit=5` with U1's own token.
3. *(expect)* 200 — D3 recommended (because U2 rode with D1 and D3, score=1) and D4 recommended (because U3 rode with D2 and D4, score=1). Each result enriched with name and vehicleType from driver-service. NOT including D1 or D2 (already ridden).
4. *(verify)* Feign call to user-service for ownership check (404 path). Feign calls to driver-service for each candidate driver. No SQL on user-postgres or driver-postgres from ride-service.

Features Verified as NOT Cross-Service (Uber-Specific)

- **S3-F1** (Get Rides by Status and Date Range) — Local query on `rides`. No M3 change.
- **S3-F3** (Get Fare Estimate) — Reads only the local `rides` table (counting active rides nearby for surge multiplier via `SELECT COUNT(*) FROM rides WHERE pickupLatitude BETWEEN ? AND ? AND pickupLongitude BETWEEN ? AND ? AND status IN (...)`). The `rides` table is owned by ride-service. No M3 change.
- **S3-F5** (Filter Rides by Metadata Field) — Local JSONB query on `rides`. No M3 change.
- **S3-F6** (Ride Analytics by Time Period — M1) — Aggregates totalRides, completedRides, cancelledRides, totalRevenue, averageFare, completionRate from the local `rides` table. M1 uses `rides.fare` for `totalRevenue` (local column), not `payments.amount`. No M3 change.
- **S3-F8** (Add Stops to Existing Ride) — Local writes to `rides` + `ride_stops` (both intra-service). No M3 change.
- **S3-F9** (Get Ride Details with Stops) — Local read of `rides` + `ride_stops`. No M3 change.
- **S3-F10** (Get Ride Analytics Dashboard — M2) — M2's Behavior step 3 reads "sum of payments.amount across COMPLETED rides" via a `rides JOIN payments` shared-DB query. In M3, ride-service computes `totalRevenue` locally as `sum(rides.fare)` across COMPLETED rides — the M1 Ride entity already stores `fare` on its own table. The slight accuracy delta vs `payments.amount` (which after M2's S5-F10 carries an additive `surgeFee` JSONB key) is acceptable for an analytics dashboard: the dashboard reports gross ride fare, not platform-net revenue. The other DTO fields (totalRides, averageRideFare, completionRate, ridesByStatus) all derive from the local `rides` table. The MongoDB `ANALYTICS_VIEWED` log

+ 10-minute Redis cache from M2 stay in place. No M3 cross-service refactor required.

RabbitMQ: S3 Publishes

Routing key	Exchange	Payload	When
`ride.placed`	`ride.events`	`{rideId, userId, driverId}`	After S3-F2 assigns a driver
`ride.completed`	`ride.events`	`{rideId, userId, driverId, fare}`	After S3-F4 completion (saga trigger)
`ride.cancelled`	`ride.events`	`{rideId, userId, driverId, reason}`	After S3-F7 cancel, or after compensation

RabbitMQ: S3 Consumes

Routing key	From exchange	Action
`user.registered`	`user.events`	Audit log only — record user account creation in ride-service's MongoDB `ride_events`. No state mutation in ride-postgres (ride rows are created lazily on first ride request).
`user.deactivated`	`user.events`	Audit log only — record user deactivation timestamp. No compensation is needed: S1-F4's Feign pre-check guarantees the user has zero active rides at deactivation time, so there is nothing to cancel.
`payment.initiated`	`payment.events`	Mark ride status = PAYMENT_PENDING
`payment.completed`	`payment.events`	Mark ride status = PAID
`payment.failed`	`payment.events`	Mark ride status = PAYMENT_FAILED → publish `ride.cancelled` (compensation trigger)
`payment.refunded`	`payment.events`	Mark ride status = REFUNDED

Queue declaration: `ride.saga-feedback` with DLQ `ride.saga-feedback.dlq`.

S3 Deliverables

- Expose `GET /api/rides/user/{userId}/summary`, `active-count`, `completed-count`
- Expose `GET /api/rides/driver/{driverId}/summary`, `active-count`, `completed-count`
- `feign.driver-service.url`, `feign.user-service.url`, `feign.location-service.url` in application.yml
- `DriverServiceClient` with `getDriver`, `getDriverAvailability`
- `UserServiceClient` with `getUser`
- `LocationServiceClient` with `getRecentLocationForDriver`
- S3-F2 refactored to use Feign → driver-service; publishes `ride.placed`
- S3-F4 refactored: pre-saga Feign checks + publish `ride.completed` (no direct payment insert, no direct driver status update)
- S3-F7 refactored: publish `ride.cancelled` (no direct driver status update)
- S3-F11 refactored to use Feign → user-service + driver-service
- S3-F12 refactored to use Feign → user-service + driver-service
- New Ride status values added to enum: `PAYMENT_PENDING`, `PAID`, `PAYMENT_FAILED`, `REFUNDED`
- RabbitMQ `ride.events` TopicExchange declared
- Consumers for `payment.initiated`, `payment.completed`, `payment.failed`, `payment.refunded`
- `logback-spring.xml` with Loki4J appender

Section 6 — Location Service Refactoring (S4)

New Endpoints S4 Must Expose

Endpoint	Called by	Returns	Description
`GET /api/locations/driver/{driverId}/recent`	S3 (saga pre-check)	`LocationDTO`	Returns the most recent location ping for this driver, only if its timestamp is within the last 5 minutes. 404 if none recent. New endpoint not in M1/M2.

This endpoint is the saga's "active sub-entity exists" pre-check (see §8.3). A ride cannot be marked COMPLETED unless the driver has a fresh GPS ping — proving the driver is online and tracking is functional.

[S4-F3] Find Nearby Available Drivers

- **Branch:** `feat/M3/location/S4-F3/<studentID>`
- **Endpoint:** `GET /api/locations/nearby?lat={lat}&lon={lon}&radiusKm={r}`

M1 implementation: Native SQL `JOIN locations JOIN drivers ON locations.driver_id = drivers.id WHERE drivers.status = 'AVAILABLE'` on the shared database, then filtering by Euclidean distance from the requested point.

M3 change: location-service cannot JOIN the `drivers` table (it lives in driver-postgres). Instead:

1. Compute the candidate set locally: query the latest location per driver in location-postgres, filter by Euclidean distance ≤ radiusKm. This produces a list of `(driverId, latitude, longitude, distanceKm)` tuples (cap at 100 — see §2.12).
2. For each candidate, call Feign → driver-service `GET /api/drivers/{id}` to fetch `name` and `status`.
3. Drop candidates whose status ≠ AVAILABLE.
4. Build `List<NearbyDriverDTO>` with driverId, driverName, latitude, longitude, distanceKm, sorted ascending by distance. Default pagination: `?page=0&size=20` (max size=100).

Test scenario:

1. *(setup)* 3 drivers in driver-postgres: Driver A (status=AVAILABLE), Driver B (status=AVAILABLE), Driver C (status=BUSY). In location-postgres: latest locations Driver A (30.04, 31.23), Driver B (30.05, 31.24), Driver C (30.05, 31.24).
2. *(action)* `GET /api/locations/nearby?lat=30.044&lon=31.235&radiusKm=5`.
3. *(expect)* 200 — returns Driver A and Driver B sorted by distance. Driver C excluded because its status is BUSY.
4. *(verify)* Feign calls to driver-service for each candidate. No JOIN on driver-postgres from location-service.

[S4-F9] Find Stationary Drivers

- **Branch:** `feat/M3/location/S4-F9/<studentID>`
- **Endpoint:** `GET /api/locations/stationary?maxSpeed={s}&sinceMinutes={m}`

M1 implementation: Native SQL `JOIN locations WITH drivers` using a subquery for the latest location per driver, filtering by JSONB speed ≤ maxSpeed and timestamp within sinceMinutes.

M3 change: Replace the JOIN with Feign calls.

1. Local query on location-postgres: latest location per driver, filtered by `metadata->>'speed' <= maxSpeed` (cast to numeric) and timestamp within `now() - sinceMinutes` minutes, sorted ascending by lastUpdated. Return `(driverId, latitude, longitude, lastSpeed, lastUpdated)` tuples (cap at 100 — see §2.12).
2. For each driverId, call Feign → driver-service `GET /api/drivers/{id}` to fetch `name`.

3. Build `List<StationaryDriverDTO>` with driverId, driverName, latitude, longitude, lastSpeed, lastUpdated. Default pagination: `?page=0&size=20` (max size=100).

Test scenario:

1. *(setup)* 3 drivers in driver-postgres. In location-postgres: Driver A latest speed=0 (10 minutes ago), Driver B latest speed=5 (10 minutes ago), Driver C latest speed=50 (10 minutes ago).
2. *(action)* `GET /api/locations/stationary?maxSpeed=10&sinceMinutes=30`.
3. *(expect)* 200 — returns Driver A and Driver B (speed ≤ 10), with names enriched via Feign. Driver C excluded.
4. *(action)* `GET /api/locations/stationary?maxSpeed=0&sinceMinutes=30` → only Driver A.
5. *(verify)* No JOIN across location-postgres and driver-postgres.

Features Verified as NOT Cross-Service (Uber-Specific)

- **S4-F1, S4-F2, S4-F4, S4-F8 (driver-existence checks)** — In M1 these features verified the driver exists via a native SQL count on the shared `drivers` table before reading/writing locations. In M3 the existence check is dropped from the request path: the location-service consumes `user.registered`-style events from driver-service is not implemented (driver registration is not part of the saga). Instead, since the Bearer token ownership rules in M2 already gate "who can update which driver's location," and location records are append-only telemetry, an orphaned driverId in `locations` is acceptable at M2/M3 scope. No M3 change to these endpoints beyond removing the SQL existence check that previously hit the shared `drivers` table.
- **S4-F5** (Filter Locations by Metadata) — Local JSONB query on `locations`. No M3 change.
- **S4-F6** (Get Locations in Date Range) — Local query on `locations`. No M3 change.
- **S4-F7** (Purge Old Location Data) — Local DELETE on `locations`. No M3 change.
- **S4-F10** (Get Location Analytics Dashboard — M2) — Aggregates only the local `locations` table. No M3 change.
- **S4-F11** (Record Location GPS Event — M2) — In M2 it verified the driver via shared SQL; in M3 the existence check is dropped on the same rationale as S4-F1/S4-F2. The Cassandra write + MongoDB Observer log are local-service operations.
- **S4-F12** (Get Location Tracking Timeline — M2) — Reads Cassandra. No M3 change.

RabbitMQ: S4 Publishes

Routing key	Exchange	Payload	When
`location.tracked`	`location.events`	`{driverId, rideId, latitude, longitude}`	Optional — emit on every S4-F11 tracking event for audit only

RabbitMQ: S4 Consumes

Routing key	From exchange	Action
`ride.completed`	`ride.events`	Mark the most recent location for this driver with the rideId + completion timestamp (final ping for the trip); write a `TRIP_COMPLETED` entry to `location_events`
`ride.cancelled`	`ride.events`	Write a `TRIP_CANCELLED` entry to `location_events` for audit; no Cassandra mutation

Queue declaration: `location.ride.saga-listener` with DLQ `location.ride.saga-listener.dlq`.

S4 Deliverables

- Implement `GET /api/locations/driver/{driverId}/recent` — returns latest location only if within last 5 minutes, 404 otherwise
- `feign.driver-service.url` in location-service application.yml
- `DriverServiceClient` Feign interface with `getDriver`
- S4-F3 refactored to use Feign → driver-service for driver name + AVAILABLE filter
- S4-F9 refactored to use Feign → driver-service for driver name enrichment
- RabbitMQ `location.events` TopicExchange declared (optional `location.tracked` publishes)
- Consumer for `ride.completed`, `ride.cancelled` with auto ACK + DLQ via `x-dead-letter-exchange`
- `logback-spring.xml` with Loki4J appender

Section 7 — Payment Service Refactoring (S5)

New Endpoints S5 Must Expose

Endpoint	Called by	Returns	Description
`GET /api/payments/user/{userId}/total?startDate={d}&endDate={d}`	S1 (S1-F6)	`BigDecimal`	Total COMPLETED payment amount for this user in the date range. 0.0 if no payments.

[S5-F3] User Payment Summary

- **Branch:** `feat/M3/payment/S5-F3/<studentID>`
- **Endpoint:** `GET /api/payments/user/{userId}/summary`
- **Auth:** ownership rule (§2.10) — `userId` must equal `X-User-Id` or caller must be ADMIN.

M1 implementation: `SELECT COUNT(*) FROM users WHERE id = ?` on the shared database to verify the user exists before returning payment data.

M3 change: Replace the user existence check with Feign → user-service `GET /api/users/{id}`. 404 from Feign → throw 404. Otherwise build the breakdown from the local `payments` table as in M1. Empty-result format: if the user has no COMPLETED payments, return `{totalPayments: 0, totalAmount: 0.00, methodBreakdown: {}}` (200, not 404 — the user exists but has no payments).

Test scenario:

1. *(setup)* User ID=1 in user-postgres. 4 COMPLETED payments in payment-postgres for userId=1: 2 CREDIT_CARD (75, 120), 1 CASH (50), 1 WALLET (30).
2. *(action)* `GET /api/payments/user/1/summary` with `X-User-Id: 1`.
3. *(expect)* 200 — totalPayments=4, totalAmount=275, methodBreakdown=`{CREDIT_CARD:195, CASH:50, WALLET:30}`.
4. *(action)* `GET /api/payments/user/999/summary` → Feign → user-service throws 404 → 404.
5. *(action)* `GET /api/payments/user/1/summary` with `X-User-Id: 2` (a different non-admin user) → 403.
6. *(action)* User exists but has zero payments → 200 with empty methodBreakdown.

[S5-F4] Process Payment for Ride

- **Branch:** `feat/M3/payment/S5-F4/<studentID>`
- **Endpoint:** `POST /api/payments/ride/{rideId}`
- **Body:** `{"method": String, "cardLastFour": String}`

M1 implementation: `SELECT * FROM rides WHERE id = ?` on the shared database to validate the ride exists and is COMPLETED. Then updates the existing PENDING payment row that S3-F4 created when the ride was completed.

M3 change: Replace the ride lookup with Feign → ride-service `GET /api/rides/{rideId}`. Validate:

- 404 from Feign → throw 404
- status ≠ PAYMENT_PENDING → throw 400 (“Ride is not awaiting payment”)
- A PENDING Payment row must already exist locally in payment-postgres (created by the saga’s `ride.completed` consumer); if missing → throw 400 (“No pending payment for this ride”)

If valid, attempt to process the payment (in M2 this was a synchronous mock):

- **On success:** update the existing Payment row to status=COMPLETED, populate `transactionDetails` JSONB (`gatewayResponse=approved`, `cardLastFour`, plus the `surgeFee` key per Section 4.6 of the M2 spec). Publish `payment.completed` to `payment.events`.
- **On failure:** set the Payment row to status=FAILED. Publish `payment.failed` to `payment.events`.

Test scenario:

1. *(setup)* Ride ID=1 in ride-postgres: status=PAYMENT_PENDING, fare=200. PENDING Payment in payment-postgres for rideId=1, amount=200.
2. *(action)* `POST /api/payments/ride/1` body `{"method": "CREDIT_CARD", "cardLastFour": "4242"}`.
3. *(expect)* 201 — payment status=COMPLETED. `payment.completed` event published. After consumption: ride status=PAID.
4. *(verify)* Feign call to ride-service to validate status=PAYMENT_PENDING. No direct query on ride-postgres.
5. *(action)* Ride status=ACCEPTED (not PAYMENT_PENDING) → Feign returns ride → 400.
6. *(action)* Body with an unsupported method (`{"method": "BITCOIN"}`) → 400 + `payment.failed` event published.
7. *(action)* `POST /api/payments/ride/999` body `{"method": "CREDIT_CARD"}` → Feign → ride-service returns 404 → payment-service throws 404.

[S5-F10] Get Fare Revenue by Vehicle Type with Surge Breakdown (M2)

- **Branch:** `feat/M3/payment/S5-F10/<studentID>`
- **Endpoint:** `GET /api/payments/analytics/vehicle-type?startDate={date}&endDate={date}`

M2 implementation: Three-table JOIN: `payments JOIN rides ON rides.id = payments.ride_id JOIN drivers ON drivers.id = rides.driver_id` — reads `drivers.vehicleDetails->>'vehicleType'` from the shared database to group revenue by vehicle type.

M3 change: Two Feign-call rounds replace the JOIN:

1. Fetch all COMPLETED payments in date range from payment-postgres (local query, no JOIN; cap at 100 distinct rideIds — see §2.12).
2. For each payment, Feign → ride-service `GET /api/rides/{rideId}` → get `driverId`.
3. For each `driverId`, Feign → driver-service `GET /api/drivers/{id}` → read `vehicleDetails.vehicleType`.
4. Group by `vehicleType`, aggregate `surgeFeeRevenue` (sum of `transactionDetails->>'surgeFee'`, defaulting to 15% of amount when the key is missing per Section 4.6 of the M2 spec), `baseFareRevenue = amount - surgeFee`, `totalRevenue = sum(amount)`, `rideCount = count(distinct rideId)`.

Optimization: Cache the `driverId → vehicleType` lookup locally (in a Map) within the request lifecycle to avoid calling driver-service once per payment when many payments share the same driver. This collapses the N+1 fan-out in step 3 from O(payments) to O(distinct drivers).

Test scenario:

1. *(setup)* Drivers in driver-postgres: D1 (SEDAN), D2 (SUV), D3 (SEDAN). In ride-postgres: 3 rides on D1/D3 (SEDAN), 2 rides on D2 (SUV). In payment-postgres: SEDAN total=600 (surgeFee=90), SUV total=400 (surgeFee=60).
2. *(action)* `GET /api/payments/analytics/vehicle-type?startDate=2026-03-01&endDate=2026-03-31`.
3. *(expect)* 200 — `[{vehicleType: SEDAN, baseFareRevenue: 510, surgeFeeRevenue: 90, totalRevenue: 600, rideCount: 3}, {vehicleType: SUV, baseFareRevenue: 340, surgeFeeRevenue: 60, totalRevenue: 400, rideCount: 2}]`.
4. *(verify)* No JOIN across payment-postgres, ride-postgres, and driver-postgres. Two rounds of Feign calls.

Features Verified as NOT Cross-Service (Uber-Specific)

- **S5-F1** (Get Payments by Status and Date Range) — Local query on `payments`. No M3 change.
- **S5-F2** (Process Refund) — Local update on `payments`. No M3 change.
- **S5-F5** (Apply Coupon to Payment) — Operates on `payments`, `coupons`, `payment_coupons` — all in payment-service. No M3 change.
- **S5-F6** (Revenue Report by Date Range) — Local aggregation on `payments`. No M3 change.
- **S5-F7** (Retry Failed Payment) — Local update on `payments`. No M3 change.
- **S5-F8** (Payment Details with Coupons) — Local read of `payments` + `payment_coupons` + `coupons`. No M3 change.
- **S5-F9** (Most Used Coupons Report) — Local aggregation on `payment_coupons` + `coupons`. No M3 change.

- **S5-F11** (Payment Method Breakdown — M2) — Reads MongoDB `payment_audit_trail`. No M3 change.
- **S5-F12** (Process Ride Refund with Surge Handling — M2) — Local update on `payments` + Strategy pattern + MongoDB write. No M3 change.

RabbitMQ: S5 Publishes

Routing key	Exchange	Payload	When
`payment.initiated`	`payment.events`	`{paymentId, rideId, amount}`	After consuming `ride.completed` and creating a PENDING payment
`payment.completed`	`payment.events`	`{paymentId, rideId, amount}`	After S5-F4 successfully processes payment
`payment.failed`	`payment.events`	`{paymentId, rideId, reason}`	After S5-F4 fails to process payment
`payment.refunded`	`payment.events`	`{paymentId, rideId, refundAmount}`	After consuming `ride.cancelled` and refunding the payment

RabbitMQ: S5 Consumes

Routing key	From exchange	Action
`ride.completed`	`ride.events`	Create a PENDING Payment in payment-postgres with rideId, userId, amount=fare, status=PENDING. Publish `payment.initiated`.
`ride.cancelled`	`ride.events`	If a PENDING/COMPLETED Payment exists for this ride → run the M2 S5-F12 Strategy refund logic to compute the refund amount → set status=REFUNDED → publish `payment.refunded`

Queue declaration: `payment.saga-listener` with DLQ `payment.saga-listener.dlq`.

S5 Deliverables

- Expose `GET /api/payments/user/{userId}/total?startDate=&endDate=`
- `feign.user-service.url`, `feign.ride-service.url`, `feign.driver-service.url` in application.yml
- `UserServiceClient` Feign interface with `getUser`
- `RideServiceClient` Feign interface with `getRide`
- `DriverServiceClient` Feign interface with `getDriver`
- S5-F3 refactored to use Feign → user-service for user existence check
- S5-F4 refactored to use Feign → ride-service for ride status validation; publishes `payment.completed` or `payment.failed`
- S5-F10 refactored to use Feign → ride-service + driver-service for vehicle type breakdown
- RabbitMQ `payment.events` TopicExchange declared
- Consumer for `ride.completed`: create PENDING payment, publish `payment.initiated`
- Consumer for `ride.cancelled`: refund if payment exists, publish `payment.refunded`
- `logback-spring.xml` with Loki4J appender

Section 8 — Ride Lifecycle Saga & Cancellation Cascade

8.1 What Is a Choreography Saga

When a business transaction spans multiple services, there is no distributed rollback. The Choreography Saga achieves eventual consistency through:

- **Forward path:** each service listens for the previous step's success event and executes its part.
- **Compensation path:** on failure, the failing service publishes a failure event; every service that already committed reverses its local change on receipt of the compensation event.

For Uber, the saga binds the ride lifecycle to the payment lifecycle: a rider/driver completes the ride, payment is settled asynchronously, and a failed settlement reverses every committed side effect (driver freed, stats reversed, payment refunded).

8.2 Saga Overview — All 5 Services

The saga is triggered by `PUT /api/rides/{id}/complete` (S3-F4). The saga proceeds through these steps:

1. **Saga trigger** — Driver initiates ride completion.
2. **Ride completed** — Ride transitions IN_PROGRESS → COMPLETED.
3. **Forward fan-out** — `ride.completed` event published, consumed by S1, S2, S4, S5.
4. **Awaiting payment** — Ride transitions COMPLETED → PAYMENT_PENDING.
5. **Customer payment** — Rider triggers `POST /api/payments/ride/{rideId}`. 6a. **Outcome · Success** — Ride transitions PAYMENT_PENDING → PAID. 6b. **Outcome · Failure** — Ride transitions PAYMENT_PENDING → PAYMENT_FAILED.
6. **Compensation cascade** — Failed payment triggers reversal across all services.
7. **Refunded** — Ride transitions PAYMENT_FAILED → REFUNDED.

8.3 S3-F4 — Complete Ride (Saga Trigger)

- **Branch:** `feat/M3/ride/S3-F4/<studentID>`
- **Endpoint:** `PUT /api/rides/{id}/complete`

M1 implementation: Set status = COMPLETED + completedAt. Calculate fare if unset. Update the assigned driver's status back to AVAILABLE via native SQL on the shared `drivers` table. Create a PENDING Payment row directly in the shared `payments` table.

M3 change: Remove both the direct driver update and the direct payment insert. Run three Feign pre-checks, then publish `ride.completed`.

Behavior:

1. Find ride by ID → 404 if not found.
2. Validate status = IN_PROGRESS → 400 if not.
3. Calculate fare if unset using the M1 surge-pricing formula (locally — surge counter reads `rides` only, no cross-service call).
4. **Pre-saga Feign checks** (all three must pass before any event is published):
   - Feign → user-service `GET /api/users/{id}` → status must be ACTIVE; if 404 or DEACTIVATED → 400
   - Feign → driver-service `GET /api/drivers/{id}` → status must be BUSY (proves the driver is currently assigned and active for this ride); if 404 or any other status → 400
   - Feign → location-service `GET /api/locations/driver/{driverId}/recent` → returns 200 with the latest GPS ping if it is within the last 5 minutes; 404 → 400 ("driver not actively tracked")
5. Mark ride status = COMPLETED, set completedAt = now(), save.

6. Publish `ride.completed` to `ride.events` exchange with payload `{rideId, userId, driverId, fare}`.
7. Return 200 with the updated ride.

Ride transitions from COMPLETED → PAYMENT_PENDING **asynchronously** when S3 consumes back the `payment.initiated` event from payment-service.

Test scenario:

1. *(setup)* Ride ID=1 in ride-postgres: status=IN_PROGRESS, driverId=5, userId=10, fare=null. Driver ID=5 in driver-postgres: status=BUSY. User ID=10 in user-postgres: status=ACTIVE. Location in location-postgres: driverId=5, timestamp = 2 minutes ago.
2. *(action)* `PUT /api/rides/1/complete`.
3. *(expect)* 200 — ride status=COMPLETED, completedAt set, fare computed. `ride.completed` published to `ride.events`.
4. *(verify)* No direct UPDATE drivers or INSERT INTO payments from ride-service.
5. *(action)* Same ride, but driver-service returns OFFLINE → 400. No event published.
6. *(action)* User status=DEACTIVATED → Feign → user-service returns DEACTIVATED → 400. No event published.
7. *(action)* location-service returns 404 (no recent ping) → 400. No event published.

8.4 S3-F7 — Cancel Ride

- **Branch:** `feat/M3/ride/S3-F7/<studentID>`
- **Endpoint:** `PUT /api/rides/{id}/cancel`

M1 implementation: Set ride status = CANCELLED. If a driver was already assigned, run `UPDATE drivers SET status = 'AVAILABLE' WHERE id = ?` directly on the shared database.

M3 change: Remove the direct `drivers` write. Publish `ride.cancelled` — driver-service consumes it and flips its own driver row to AVAILABLE; payment-service consumes it and refunds any pre-existing payment.

Behavior:

1. Find ride by ID → 404 if not found.
2. Validate status IN (REQUESTED, ACCEPTED) → 400 if not.
3. Set ride status = CANCELLED.
4. Publish `ride.cancelled` to `ride.events` exchange with payload `{rideId, userId, driverId, reason: "user_requested"}`. If driverId is null (cancelled before assignment), the payload still contains `driverId=null` — driver-service silently ignores when it sees null.
5. Return 200.

Driver re-availability and any payment refund happen asynchronously — driver-service and payment-service each consume `ride.cancelled` and update their own databases.

Test scenario:

1. *(setup)* Ride ID=1 in ride-postgres: status=ACCEPTED, driverId=5, userId=10. Driver ID=5 in driver-postgres: status=BUSY.
2. *(action)* `PUT /api/rides/1/cancel`.
3. *(expect)* 200 — ride status=CANCELLED. `ride.cancelled` published.
4. *(verify)* No direct UPDATE drivers from ride-service. After event processing: driver-postgres has driver 5 status=AVAILABLE.
5. *(action)* Try cancelling a COMPLETED ride → 400.
6. *(action)* Try cancelling an IN_PROGRESS ride → 400.
7. *(action)* `PUT /api/rides/999/cancel` → 404 (ride not found). No event published.

8.5 Saga Participant Summary

Service	Feign calls in saga	Publishes	Consumes
user-service	Target of S3 + S5 pre-checks	`user.registered`, `user.deactivated`	`ride.completed`, `ride.cancelled`
driver-service	Target of S3 pre-check + S4 enrichment	`driver.status-changed`, `driver.rated`, `driver.document.verified`	`ride.placed`, `ride.completed`, `ride.cancelled`
ride-service	→ user-service (pre-check), → driver-service (pre-check), → location-service (pre-check)	`ride.placed`, `ride.completed`, `ride.cancelled`	`payment.initiated`, `payment.completed`, `payment.failed`, `payment.refunded`
location-service	Target of S3 pre-check; → driver-service (S4-F3, S4-F9 enrichment)	`location.tracked` (audit only, optional)	`ride.placed`, `ride.completed`, `ride.cancelled`
payment-service	→ user-service (S5-F3 existence), → ride-service (S5-F4, S5-F10), → driver-service (S5-F10)	`payment.initiated`, `payment.completed`, `payment.failed`, `payment.refunded`	`ride.completed`, `ride.cancelled`

8.6 Saga Test Scenarios

Scenario A — Happy path end-to-end

1. *(setup)* In respective databases: User ID=1 (ACTIVE), Driver ID=5 (BUSY, assigned to Ride ID=10), Ride ID=10 (status=IN_PROGRESS, userId=1, driverId=5, fare=null), Location for driver 5 with timestamp 1 minute ago.
2. *(action)* `PUT /api/rides/10/complete` → all three pre-checks pass.
3. *(expect)* 200. Ride status = COMPLETED. `ride.completed` published with fare computed.
4. *(verify after event processing)* Ride status = PAYMENT_PENDING; driver-postgres driver 5 status = AVAILABLE with totalEarnings incremented; payment-postgres has a PENDING Payment for rideId=10 with amount = fare.
5. *(action)* `POST /api/payments/ride/10` body `{"method": "CREDIT_CARD", "cardLastFour": "4242"}`.
6. *(expect)* 201. `payment.completed` published.
7. *(verify after event processing)* Ride status = PAID. (Poll `GET /api/rides/10` until status changes from PAYMENT_PENDING to PAID, or wait ≥ 1s for the ride-service consumer of `payment.completed` to run.)

Scenario B — Payment failure and compensation

1. *(setup)* Same as Scenario A — reach Ride status = PAYMENT_PENDING with PENDING payment in payment-postgres.
2. *(action)* `POST /api/payments/ride/10` body `{"method": "BITCOIN"}` (deliberately unsupported method to force failure).
3. *(expect)* 400. `payment.failed` published.
4. *(verify after the compensation cascade runs)* The cascade is 5 hops async: `payment.failed` → S3 sets Ride = PAYMENT_FAILED → S3 publishes `ride.cancelled` (reason="payment_failed") → S1/S2/S4/S5 consumers reverse their state → S5 issues refund and publishes `payment.refunded` → S3 sets Ride = REFUNDED. Poll `GET /api/rides/10` until status = REFUNDED, or wait ≥ 3s for the full cascade. Then assert: `ride.cancelled` was published with reason="payment_failed"; rider stats reversed; driver stats reversed (driver back to AVAILABLE if still BUSY); payment-postgres Payment row status = REFUNDED; ride-postgres Ride status = REFUNDED.

Scenario C — Pre-check failure (no recent location ping)

1. *(setup)* User ID=1 (ACTIVE), Driver ID=5 (BUSY), Ride ID=10 (IN_PROGRESS). Latest location for driver 5 has a timestamp from 30 minutes ago (stale).
2. *(action)* `PUT /api/rides/10/complete`.
3. *(expect)* 400 — location-service `GET /api/locations/driver/5/recent` returns 404 because the latest ping is

older than 5 minutes. S3 aborts before publishing any event.

4. *(verify)* No `ride.completed` event in RabbitMQ. Ride status still = IN_PROGRESS. Driver stays BUSY.

8.7 Saga Infrastructure Deliverables

- `RideEventConfig` in ride-service: `ride.events` TopicExchange
- `DriverEventConfig` in driver-service: `driver.events` TopicExchange
- `UserEventConfig` in user-service: `user.events` TopicExchange
- `LocationEventConfig` in location-service: `location.events` TopicExchange (publisher optional)
- `PaymentEventConfig` in payment-service: `payment.events` TopicExchange
- All consumer queue declarations with DLQ (one per service per exchange it listens to)
- All event payload record classes (e.g., `RideCompletedEvent`, `PaymentFailedEvent`)
- Saga test scenarios A, B, C verified end-to-end

Section 9 — Spring Cloud Gateway

9.1 New Maven Module

Add `api-gateway` as the 7th module in the root `pom.xml` (after contracts and the 5 services):

```
<modules>
    <module>contracts</module>
    <module>user-service</module>
    <module>driver-service</module>
    <module>ride-service</module>
    <module>location-service</module>
    <module>payment-service</module>
    <module>api-gateway</module>
</modules>
```

The gateway runs on port 8080 internally, exposed as NodePort 30080 externally.

Dependencies:

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway-server-webflux</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

Spring Cloud Gateway is reactive (Project Reactor). **Do NOT add** `spring-boot-starter-web` — it conflicts with webflux. The artifact is `spring-cloud-starter-gateway-server-webflux` (the renamed gateway starter introduced in Spring Cloud 2025.1.x); the old `spring-cloud-starter-gateway` artifact name is from the 2025.0.x release train and does not resolve under the BOM declared in §2.1.

9.2 Routing Configuration

```
spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: http://user-service:8080
          predicates:
            - Path=/api/users/**, /api/auth/**
        - id: driver-service
          uri: http://driver-service:8080
          predicates:
            - Path=/api/drivers/**
        - id: ride-service
          uri: http://ride-service:8080
          predicates:
            - Path=/api/rides/**
        - id: location-service
          uri: http://location-service:8080
          predicates:
            - Path=/api/locations/**
        - id: payment-service
          uri: http://payment-service:8080
          predicates:
            - Path=/api/payments/**
```

9.3 JWT Global Filter

The api-gateway runs Spring Cloud Gateway (reactive WebFlux), so the M2 servlet-based JWT filter must be **rewritten** for the reactive API — not just copy-pasted. The five concrete differences:

1. **Class shape.** Implement `org.springframework.cloud.gateway.filter.GlobalFilter` instead of `OncePerRequestFilter`. The signature becomes `Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain)` — there is no `HttpServletRequest` / `HttpServletResponse`, no void return, and no `chain.doFilter(req, res)`.
2. **Filter ordering.** Annotate the class with `@Component` AND implement `org.springframework.core.Ordered` returning `-1` from `getOrder()` (or use `@Order(-1)`) so it executes **before** Spring Cloud Gateway's route-resolution filter. M2 had no equivalent because servlet filters chain by `web.xml` / `FilterRegistrationBean` order.
3. **Path bypass.** Read the request path via `exchange.getRequest().getPath().value()` (reactive) — not `request.getRequestURI()` (servlet). Bypass `/api/auth/**` (register, login, refresh) without invoking the JWT validator. All other paths require a valid `Authorization: Bearer <token>`.
4. **Header parsing & validation.** Read `Authorization` from `exchange.getRequest().getHeaders().getFirst("Authorization")`, strip the `Bearer` prefix, validate against the env-injected `JWT_SECRET` (see §10.3 ConfigMap) using the same JJWT library M2 used. On token decode failure or expiry → return 401 by setting `exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED)` and returning `exchange.getResponse().setComplete()` — a terminal `Mono<Void>` that short-circuits the chain.
5. **Header forwarding.** On successful validation, mutate the downstream request so each service's M2 JWT filter (still in place as defense-in-depth) sees the resolved identity:

```
        ServerHttpRequest mutated = exchange.getRequest().mutate()
            .header("X-User-Id", claims.get("uid", Long.class).toString())
            .header("X-User-Role", claims.get("role", String.class))
            .header("X-Correlation-ID", correlationId)
            .build();
        return chain.filter(exchange.mutate().request(mutated).build());
```

The token-validation logic (HMAC verification, claim extraction, expiry check) from M2's `AuthenticationService` can be lifted into a new `api-gateway/src/main/java/com/<teamID>/<domain>/gateway/auth/JwtValidator.java` — copy the JJWT-based methods directly. The api-gateway does not depend on the contracts Maven module (per §12.1) and does not call any other service via Feign, so the validator is self-contained.

9.4 Gateway Deliverables

- `api-gateway` Maven module created and added to root `pom.xml`
- `spring-cloud-starter-gateway-server-webflux` + `spring-boot-starter-webflux` dependencies
- All 5 service route entries in `application.yml`
- `JwtGatewayFilter` implemented as `GlobalFilter` (Spring Cloud Gateway reactive interface), registered as `@Component`, with `Ordered.getOrder() == -1` so it runs before Spring Cloud Gateway's route-resolution filter
- `/api/auth/**` bypass (no JWT check on register/login)
- `X-User-Id`, `X-User-Role`, `X-Correlation-ID` headers forwarded to downstream services
- `docker-compose.yml` updated: per-service postgres containers + RabbitMQ container + api-gateway service

Section 10 — Kubernetes Deployment

10.1 Directory Structure

```
k8s/
├── namespaces/
│   └── namespace.yaml                  # namespace: uber
├── secrets/
│   ├── jwt-secret.yaml
│   ├── user-postgres-secret.yaml
│   ├── driver-postgres-secret.yaml
│   ├── ride-postgres-secret.yaml
│   ├── location-postgres-secret.yaml
│   └── payment-postgres-secret.yaml
├── configmaps/
│   ├── user-service-configmap.yaml
│   ├── driver-service-configmap.yaml
│   ├── ride-service-configmap.yaml
│   ├── location-service-configmap.yaml
│   ├── payment-service-configmap.yaml
│   └── gateway-configmap.yaml
├── pvcs/
│   ├── user-postgres-pvc.yaml
│   ├── driver-postgres-pvc.yaml
│   ├── ride-postgres-pvc.yaml
│   ├── location-postgres-pvc.yaml
│   ├── payment-postgres-pvc.yaml
│   ├── rabbitmq-pvc.yaml
│   ├── mongo-pvc.yaml
│   ├── redis-pvc.yaml
│   ├── elasticsearch-pvc.yaml
│   ├── neo4j-pvc.yaml
│   └── cassandra-pvc.yaml
├── statefulsets/
│   ├── user-postgres-statefulset.yaml
│   ├── driver-postgres-statefulset.yaml
│   ├── ride-postgres-statefulset.yaml
│   ├── location-postgres-statefulset.yaml
│   ├── payment-postgres-statefulset.yaml
│   ├── rabbitmq-statefulset.yaml
│   ├── mongo-statefulset.yaml
│   ├── redis-statefulset.yaml
│   ├── elasticsearch-statefulset.yaml
│   ├── neo4j-statefulset.yaml
│   └── cassandra-statefulset.yaml
├── deployments/
│   ├── user-service-deployment.yaml
│   ├── driver-service-deployment.yaml
│   ├── ride-service-deployment.yaml
│   ├── location-service-deployment.yaml
│   └── payment-service-deployment.yaml
├── services/
│   ├── user-service-svc.yaml           # ClusterIP
│   ├── user-postgres-svc.yaml          # headless
│   ├── driver-service-svc.yaml         # ClusterIP
│   ├── driver-postgres-svc.yaml        # headless
│   ├── ride-service-svc.yaml           # ClusterIP
│   ├── ride-postgres-svc.yaml          # headless
│   ├── location-service-svc.yaml       # ClusterIP
│   ├── location-postgres-svc.yaml      # headless
│   ├── payment-service-svc.yaml        # ClusterIP
│   ├── payment-postgres-svc.yaml       # headless
│   ├── rabbitmq-svc.yaml
│   ├── mongo-svc.yaml
│   ├── redis-svc.yaml
│   ├── elasticsearch-svc.yaml
│   ├── neo4j-svc.yaml
│   └── cassandra-svc.yaml
└── api-gateway/
    ├── gateway-deployment.yaml
    └── gateway-service.yaml             # type: NodePort (30080)
```

10.2 Namespace

```
apiVersion: v1
kind: Namespace
metadata:
  name: uber
```

All `kubectl` commands use `-n uber`.

10.3 ConfigMap Example — Ride Service

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: ride-service-configmap
  namespace: uber
data:
  SPRING_DATASOURCE_URL: jdbc:postgresql://ride-postgres:5432/uberdb-rides
  SPRING_DATASOURCE_USERNAME: user
  SPRING_RABBITMQ_HOST: rabbitmq
  FEIGN_USER_SERVICE_URL: http://user-service:8080
  FEIGN_DRIVER_SERVICE_URL: http://driver-service:8080
  FEIGN_LOCATION_SERVICE_URL: http://location-service:8080
  FEIGN_PAYMENT_SERVICE_URL: http://payment-service:8080
```

10.4 StatefulSet — Per-Service PostgreSQL (Example: ride-postgres)

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: ride-postgres
  namespace: uber
spec:
  serviceName: ride-postgres
  replicas: 1
  selector:
    matchLabels:
      app: ride-postgres
  template:
    metadata:
      labels:
        app: ride-postgres
    spec:
      containers:
        - name: postgres
          image: postgres:17
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: ride-postgres-secret
                  key: POSTGRES_USER
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: ride-postgres-secret
                  key: POSTGRES_PASSWORD
            - name: POSTGRES_DB
              valueFrom:
                secretKeyRef:
                  name: ride-postgres-secret
                  key: POSTGRES_DB
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi
```

10.5 Deployment — Spring Boot Service (Example: ride-service)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ride-service
  namespace: uber
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ride-service
  template:
    metadata:
      labels:
        app: ride-service
    spec:
      containers:
        - name: ride-service
          image: <your-registry>/ride-service:latest
          ports:
            - containerPort: 8080
          envFrom:
            - configMapRef:
                name: ride-service-configmap
            - secretRef:
                name: ride-postgres-secret
          env:
            - name: JWT_SECRET
              valueFrom:
                secretKeyRef:
                  name: jwt-secret
                  key: jwt-secret
          readinessProbe:
            httpGet:
              path: /actuator/health
              port: 8080
            initialDelaySeconds: 30
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /actuator/health
              port: 8080
            initialDelaySeconds: 60
            periodSeconds: 30
```

10.6 API Gateway NodePort Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: api-gateway
  namespace: uber
spec:
  type: NodePort
  selector:
    app: api-gateway
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 30080
```

Access the platform via: `curl http://$(minikube ip):30080/api/rides`

All other services use `type: ClusterIP`. No service other than the gateway is reachable from outside the cluster.

10.7 Deployment Order

```
kubectl apply -f k8s/namespaces/
kubectl apply -f k8s/secrets/
kubectl apply -f k8s/pvcs/
kubectl apply -f k8s/statefulsets/        # all databases first
# Wait for databases ready:
kubectl wait --for=condition=ready pod -l app=ride-postgres -n uber --timeout=120s
kubectl apply -f k8s/configmaps/
kubectl apply -f k8s/deployments/         # services after databases
kubectl apply -f k8s/services/
kubectl apply -f k8s/api-gateway/
```

10.8 Resource Limits & Healthchecks for Non-Postgres Databases

The §10.4 example only shows ride-postgres. **Every other infrastructure StatefulSet must specify both** `resources.limits.memory` (so MiniKube does not OOM under cumulative pressure) **and a database-appropriate** `livenessProbe` **+** `readinessProbe`. Carry over the M2 Docker Compose memory caps and add probes:

StatefulSet	`resources.limits.memory`	Liveness/readiness probe
`ride-postgres` (and the other 4 PG StatefulSets)	512Mi	`exec: pg_isready -U postgres` — `initialDelaySeconds: 15`, `periodSeconds: 10`
`mongodb`	512Mi	`exec: mongosh --quiet --eval "db.adminCommand('ping').ok"` — `initialDelaySeconds: 20`
`redis`	256Mi	`exec: redis-cli ping` (must return PONG) — `initialDelaySeconds: 10`, `periodSeconds: 5`
`elasticsearch`	768Mi	`httpGet: /_cluster/health?wait_for_status=yellow&timeout=1s` on port 9200 — `initialDelaySeconds: 60`
`neo4j`	768Mi	`tcpSocket` on port 7687 — `initialDelaySeconds: 30`
`cassandra`	768Mi (heap 256MB inside)	`exec: cqlsh -e 'DESCRIBE KEYSPACES'` — `initialDelaySeconds: 60`, `periodSeconds: 30`
`rabbitmq`	512Mi	`exec: rabbitmq-diagnostics -q ping` — `initialDelaySeconds: 30`, `periodSeconds: 30`

Why memory caps matter on MiniKube: the default profile starts with ~6GB. The 5 PG StatefulSets alone consume 2.5GB; add Mongo + Redis + ES + Neo4j + Cassandra + RabbitMQ + the 5 Spring Boot Deployments and you cross the cap, triggering pod evictions during grading. Caps force the JVMs and database engines to honor Kubernetes's accounting instead of trying to use all visible host RAM.

Why probes matter: `kubectl wait --for=condition=ready` (in §10.7) only signals true when the readiness probe passes. Without per-DB readiness probes, the Spring Boot services start before their datasources are accepting connections, and grader runs flake on `Connection refused`. A `tcpSocket` probe is the minimum acceptable; the per-DB exec/HTTP probes above are stronger.

The deployment-order block in §10.7 should be updated to wait for **every** database, not just `ride-postgres`:

```
for db in user-postgres driver-postgres ride-postgres location-postgres payment-postgres mongodb redis
elasticsearch neo4j cassandra rabbitmq; do
  kubectl wait --for=condition=ready pod -l app=$db -n uber --timeout=180s
done
```

K8s Deliverables

- `k8s/namespaces/namespace.yaml` — namespace `uber`

- `k8s/secrets/jwt-secret.yaml` — shared JWT secret (base64-encoded)
- 5 PostgreSQL secrets (one per service)
- 5 PostgreSQL StatefulSets with PVC templates (`postgres:17` image), `resources.limits.memory: 512Mi`, `pg_isready` probes
- 5 headless Services for PostgreSQL StatefulSets
- RabbitMQ StatefulSet + Service with `resources.limits.memory: 512Mi` and `rabbitmq-diagnostics ping` probe
- MongoDB, Redis, Elasticsearch, Neo4j, Cassandra StatefulSets + headless Services (carry over from M2 Docker Compose) with §10.8 memory caps + probes
- 5 Spring Boot Deployments with readiness/liveness probes on `/actuator/health`, `resources.limits.memory: 768Mi`
- 5 ClusterIP Services for Spring Boot services
- 6 ConfigMaps (one per service + gateway) with all env vars
- API Gateway Deployment + NodePort Service (port 30080)
- Deployment-order script waits on every DB pod (not only `ride-postgres`) before starting Spring Boot services

Section 11 — Observability

11.1 Loki4J Appender (All 5 Services)

Add to each service's `pom.xml`:

```
<dependency>
    <groupId>com.github.loki4j</groupId>
    <artifactId>loki-logback-appender</artifactId>
    <version>2.0.0</version>
</dependency>
```

Per-Service MDC Fields

Each service populates only the MDC keys relevant to its domain. `correlationId` is shared by all five services (set from the `X-Correlation-ID` header forwarded by api-gateway, or from the RabbitMQ message header in consumers). The remaining entity-specific keys differ:

Service	Entity-specific MDC keys
user-service	`userId`
driver-service	`driverId`, `rideId`, `routingKey`
ride-service	`rideId`, `userId`, `driverId`, `paymentId`, `routingKey`
location-service	`driverId`, `rideId`, `routingKey`
payment-service	`paymentId`, `rideId`, `userId`, `routingKey`

logback-spring.xml

The example below is the ride-service template (the busiest service — its JSON includes every entity field). Each other service uses the same XML structure but drops the MDC fields it does not populate from the `<message><pattern>` block.

```
<configuration>
    <appender name="LOKI" class="com.github.loki4j.logback.Loki4jAppender">
        <http>
            <url>http://loki.monitoring.svc.cluster.local:3100/loki/api/v1/push</url>
        </http>
        <format>
            <label>
                <pattern>app=uber,service=${spring.application.name},level=%level,env=k8s</pattern>
            </label>
            <message>
                <pattern>
                    {
                      "timestamp": "%d{ISO8601}",
                      "level": "%level",
                      "service": "${spring.application.name}",
                      "thread": "%thread",
                      "logger": "%logger{36}",
                      "correlationId": "%X{correlationId:-}",
                      "userId": "%X{userId:-}",
                      "driverId": "%X{driverId:-}",
                      "rideId": "%X{rideId:-}",
                      "paymentId": "%X{paymentId:-}",
                      "routingKey": "%X{routingKey:-}",
                      "message": "%msg"
                    }
                </pattern>
            </message>
        </format>
    </appender>
    <root level="INFO">
        <appender-ref ref="LOKI"/>
    </root>
</configuration>
```

MDC Population

- `correlationId` — populated by a servlet filter (`OncePerRequestFilter`) that reads the `X-Correlation-ID` header set by api-gateway and calls `MDC.put("correlationId", value)`. The filter must clear MDC in `finally`. RabbitMQ consumers must also read the `correlationId` header from the inbound `Message` and call `MDC.put` at the start of the listener method.
- Entity IDs (`userId`, `driverId`, `rideId`, `paymentId`) — populated manually by service-layer methods using `MDC.put("rideId", id.toString())` immediately before performing the operation, paired with `MDC.remove(...)` in a `finally` block to prevent leaking IDs into unrelated subsequent requests.
- `routingKey` — set by RabbitMQ publishers and consumers to the routing key being processed (e.g., `ride.completed`, `payment.failed`). This makes the Layer 3 RabbitMQ event audit panel (§11.3) usable.

Required Log Points

Each service must emit logs at the following points so the LogQL panels in §11.3 have data to query. Use SLF4J: `private static final Logger log = LoggerFactory.getLogger(<Class>.class);`.

Log point	Level	Suggested message format
Controller method entry	INFO	`"Received {} {}"` (HTTP method, path)
Controller method exit	INFO	`"Returning {} for {} {}"` (status, method, path)
Feign call — before request	INFO	`"Calling {}.{} with args={}"` (client, method, args)
Feign call — after success	INFO	`"{}.{} returned successfully"` (client, method)
Feign call — exception caught	WARN	`"Feign call to {} failed: {}"` (service, exception message)
RabbitMQ — event published	INFO	`"Published {} for {}={}"` (routingKey, entityName, id)
RabbitMQ — event consumed (start)	INFO	`"Consuming {} for {}={}"` (routingKey, entityName, id)
RabbitMQ — event processed (success)	INFO	`"Processed {} for {}={}"` (routingKey, entityName, id)
RabbitMQ — consumer error	ERROR	`"Failed to process {}: {}"` (routingKey, exception message) → DLQ
Saga state transition (S3 only)	INFO	`"Ride {} transitioning {} → {}"` (rideId, oldStatus, newStatus)
DB write success	INFO	`"{} {} saved with status={}"` (entityName, id, status)
Slow operation (> threshold)	WARN	`"Slow {} took {}ms"` (operationName, elapsedMs) — wrap operations expected to be slow under load (e.g., S5-F10 vehicle-type analytics, S2-F12 dashboard aggregation) with a stopwatch and emit when elapsed exceeds a threshold (e.g., 1000ms). Feeds the Layer 6 LogQL panel.

Required in `application.yml`:

```yaml
management:
  endpoints:
    web:
      exposure:
        include: "prometheus,health,info"
  metrics:
    distribution:
      percentiles-histogram:
        http.server.requests: true
```

11.2 Dashboard per Service

Each of the 5 services has its own Grafana dashboard. Each dashboard has at minimum **3 LogQL panels and 3 PromQL panels** chosen from the lists below. Five dashboard JSON files must be submitted (one per service).

11.3 LogQL Panel Options (choose ≥ 3 per service)

A LogQL query is built up in three layers, and every panel below uses all three:

1. **Label** — `{app="uber", service="ride-service", level="ERROR"}` — match log streams by the labels emitted by the Loki4J appender (§11.1). This narrows down which streams the rest of the query reads from.
2. **Line** — `|= "search-text"`, `!= "exclude"`, `| json`, `| line_format "{{...}}"` — filter and parse individual log lines within the matched streams. Because messages are JSON (§11.1), `| json` exposes every field (`correlationId`, `rideId`, `routingKey`, ...) for further filtering.
3. **Aggregator** — `count_over_time(...[1m])`, `rate(...[5m])`, `sum by (service) (...)` — turn the matching lines into time-series numbers that Grafana can plot.

Available Panels

1. **Error rate panel** — Count of ERROR-level log lines per service per minute. *Example purpose:* Spike detection — if ride-service logs 50 ERRORs in one minute, something is wrong.
2. **Correlation ID trace panel** — Filter all log lines by a specific `X-Correlation-ID` value across all services. *Example purpose:* Trace a single ride completion request from api-gateway through ride-service, driver-service, location-service, and payment-service.
3. **RabbitMQ event audit panel** — Lines emitted by event publishers and consumers, filtered by routing key. *Example purpose:* Show how many `ride.completed` events were published vs. how many `payment.initiated` events were consumed in the last hour.
4. **Feign call outcomes panel** — Log lines for successful Feign responses vs. `FeignException` catches. *Example purpose:* Detect when driver-service is degraded — ride-service Feign calls to it start throwing exceptions.
5. **Saga state transitions panel** — Log lines at each saga step filtered by rideId. *Example purpose:* Visualize the complete saga flow for ride ID=42: COMPLETED → PAYMENT_PENDING → PAID.
6. **Slow operation warnings panel** — Log lines where elapsed time exceeded a threshold. *Example purpose:* Alert when S5-F10 vehicle-type revenue aggregation takes > 5 seconds.

11.4 PromQL Panel Options (choose ≥ 3 per service)

A PromQL query is built up in four layers, and every panel below uses all four:

1. **Metric** — the metric name itself, e.g., `http_server_requests_seconds_count` or `jvm_memory_used_bytes`. These are exposed by each service's `/actuator/prometheus` endpoint and scraped by Prometheus every 15s.
2. **Label** — narrow the metric down with label matchers, e.g., `{service="ride-service", uri="/api/rides", method="GET"}`. Labels come from Spring Boot's Actuator metrics and from the `job_name` set in `prometheus.yml`.
3. **Range** — append a time window in square brackets, e.g., `[5m]` or `[1h]`. This turns the instant counter into a sequence of samples over that window so the function in layer 4 has data to operate on.
4. **Function** — `rate(...)`, `increase(...)`, `histogram_quantile(0.99, ...)`, `sum by (uri) (...)`, `topk(5, ...)` — converts the range vector into the final per-second rate, percentile, top-N, or grouped aggregate that Grafana plots.

Available Panels

1. **HTTP request rate panel** — Requests per second per endpoint. *Example purpose:* Which ride-service endpoints are under the most load during peak hours?
2. **HTTP latency percentiles panel** — P50/P95/P99 latency per endpoint. *Example purpose:* P99

latency on `GET /api/rides/driver/{id}/summary` is 4s — Feign enrichment is slow.

3. **JVM health panel** — Heap usage, GC pause duration, thread count. *Example purpose:* Memory pressure before OOM — location-service heap at 90% after processing 10,000 events.
4. **Database connection pool panel** — HikariCP active connections vs. pool size. *Example purpose:* Pool exhaustion alert — payment-service using 10/10 connections during saga fan-out.
5. **Cache hit/miss ratio panel** — Redis cache hits vs. misses from `cache_gets_total`. *Example purpose:* S2-F12 dashboard cache hit rate — verify caching is effective.
6. **RabbitMQ throughput panel** — Messages published vs. consumed per queue. *Example purpose:* Consumer lag on `payment.saga-listener` — published count > consumed count by > 100.

11.5 Observability Stack (K8s — `monitoring` namespace)

The three observability tools — Loki, Prometheus, and Grafana — run as their own pods inside the cluster, in a dedicated namespace called `monitoring`. Keeping them separate from the `uber` namespace means observability resources (CPU, memory, restarts) are isolated from the application services, and an issue in the app does not take down the dashboards.

The data flow uses two opposite directions:

- **Logs (push):** Each Spring Boot service runs the Loki4J appender (§11.1), which pushes log lines as JSON over HTTP to `http://loki.monitoring.svc.cluster.local:3100/loki/api/v1/push`. Loki itself never reaches into the services — they send to it.
- **Metrics (pull):** Prometheus scrapes each service's `/actuator/prometheus` endpoint on a 15-second interval (configured below). "Scrape" here just means an HTTP GET — Prometheus pulls the current metric values from each service and stores them as time-series.
- **Dashboards:** Grafana is configured with two datasources — Loki (for LogQL panels) and Prometheus (for PromQL panels). The 5 dashboard JSON files (one per service) are committed to the repo and provisioned into Grafana via a ConfigMap mount.

Component	Image	Role in the stack
Loki	`grafana/loki:2.9.4`	Receives JSON log streams pushed by Loki4J from each service.
Prometheus	`prom/prometheus:v2.51.2`	Pulls metrics from each service's `/actuator/prometheus` every 15s.
Grafana	`grafana/grafana:10.4.2`	Dashboard UI; runs the LogQL/PromQL queries from §11.3 and §11.4.

Cross-namespace DNS resolution makes this work: from `monitoring`, Prometheus reaches a service in `uber` using the fully qualified name `<service-name>.uber.svc.cluster.local`. From `uber`, services push logs to `loki.monitoring.svc.cluster.local`.

Two Namespaces Required

The cluster must contain two namespaces, each defined as its own YAML file under `k8s/namespaces/`:

Namespace	Purpose	YAML file
`uber`	All 5 application services + their PostgreSQL + RabbitMQ + NoSQL stores. Already declared in §10.2.	`k8s/namespaces/namespace.yaml`
`monitoring`	Loki + Prometheus + Grafana only. Nothing application-related deploys here.	`k8s/namespaces/monitoring-namespace.yaml`

```yaml
# k8s/namespaces/monitoring-namespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
```

Required Files & Directory Structure

All observability manifests live under `k8s/monitoring/`, separate from the application K8s tree shown in §10.1:

```
k8s/
├── namespaces/
│   ├── namespace.yaml                  # uber (already in §10.1)
│   └── monitoring-namespace.yaml       # monitoring (new — see above)
└── monitoring/
    ├── loki/
    │   ├── loki-configmap.yaml         # /etc/loki/local-config.yaml content
    │   ├── loki-pvc.yaml               # storage for log chunks (≥ 5Gi)
    │   ├── loki-statefulset.yaml       # image: grafana/loki:2.9.4, port 3100
    │   └── loki-service.yaml           # ClusterIP, port 3100 → name "loki"
    ├── prometheus/
    │   ├── prometheus-configmap.yaml   # contains prometheus.yml (scrape config below)
    │   ├── prometheus-pvc.yaml         # storage for TSDB (≥ 5Gi)
    │   ├── prometheus-deployment.yaml  # image: prom/prometheus:v2.51.2, port 9090
    │   └── prometheus-service.yaml     # ClusterIP, port 9090 → name "prometheus"
    └── grafana/
        ├── grafana-datasources.yaml    # ConfigMap — Loki + Prometheus datasource provisioning
        ├── grafana-dashboards.yaml     # ConfigMap — embeds 5 dashboard JSON files
        ├── grafana-pvc.yaml            # storage for Grafana state (≥ 1Gi)
        ├── grafana-deployment.yaml     # image: grafana/grafana:10.4.2, port 3000
        └── grafana-service.yaml        # NodePort 30030 — browser access to dashboards
```

The 5 dashboard JSON files (`user-dashboard.json`, `driver-dashboard.json`, `ride-dashboard.json`, `location-dashboard.json`, `payment-dashboard.json`) are committed to `k8s/monitoring/grafana/dashboards/` and embedded into the `grafana-dashboards.yaml` ConfigMap so Grafana auto-loads them on startup.

Required Manifests Per Component

- **Loki (StatefulSet):** Mount `loki-configmap` at `/etc/loki/`, attach the PVC at `/loki` for chunk storage. Service named `loki` so the Loki4J appender URL `http://loki.monitoring.svc.cluster.local:3100/loki/api/v1/push` resolves.
- **Prometheus (Deployment):** Mount `prometheus-configmap` at `/etc/prometheus/prometheus.yml`. The ConfigMap holds the scrape config below. Attach the PVC at `/prometheus` for the TSDB.
- **Grafana (Deployment):** Mount `grafana-datasources` at `/etc/grafana/provisioning/datasources/` and `grafana-dashboards` at `/etc/grafana/provisioning/dashboards/`. Service is `type: NodePort` on port 30030 so the dashboards are reachable from the host at `http://$(minikube ip):30030` (default credentials `admin/admin`, change on first login).

Example Manifest — Prometheus Deployment

The full file at `k8s/monitoring/prometheus/prometheus-deployment.yaml`. Loki and Grafana follow the same pattern (different image, different mount paths, different ports — see “Required Manifests Per Component” above).

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: prometheus
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels:
      app: prometheus
  template:
    metadata:
      labels:
        app: prometheus
    spec:
      containers:
        - name: prometheus
          image: prom/prometheus:v2.51.2
          args:
            - --config.file=/etc/prometheus/prometheus.yml
            - --storage.tsdb.path=/prometheus
          ports:
            - containerPort: 9090
          volumeMounts:
            - name: config
              mountPath: /etc/prometheus
            - name: storage
              mountPath: /prometheus
          readinessProbe:
            httpGet:
              path: /-/ready
              port: 9090
            initialDelaySeconds: 30
            periodSeconds: 10
      volumes:
        - name: config
          configMap:
            name: prometheus-config       # contains prometheus.yml — see scrape config below
        - name: storage
          persistentVolumeClaim:
            claimName: prometheus-pvc
```

The companion `prometheus-service.yaml` is a ClusterIP Service named `prometheus` exposing port 9090 — Grafana's Prometheus datasource uses `http://prometheus.monitoring.svc.cluster.local:9090` to reach it.

Prometheus Scrape Config

This is the file that goes inside `prometheus-configmap.yaml` under the key `prometheus.yml`:

```yaml
scrape_configs:
  - job_name: user-service
    static_configs:
      - targets: ['user-service.uber.svc.cluster.local:8080']
    metrics_path: /actuator/prometheus
  - job_name: driver-service
    static_configs:
      - targets: ['driver-service.uber.svc.cluster.local:8080']
    metrics_path: /actuator/prometheus
  - job_name: ride-service
    static_configs:
      - targets: ['ride-service.uber.svc.cluster.local:8080']
    metrics_path: /actuator/prometheus
  - job_name: location-service
    static_configs:
      - targets: ['location-service.uber.svc.cluster.local:8080']
    metrics_path: /actuator/prometheus
  - job_name: payment-service
    static_configs:
      - targets: ['payment-service.uber.svc.cluster.local:8080']
    metrics_path: /actuator/prometheus
```

Apply Order

```
kubectl apply -f k8s/namespaces/monitoring-namespace.yaml
kubectl apply -f k8s/monitoring/loki/
kubectl apply -f k8s/monitoring/prometheus/
kubectl apply -f k8s/monitoring/grafana/
kubectl wait --for=condition=ready pod -l app=loki -n monitoring --timeout=120s
kubectl wait --for=condition=ready pod -l app=prometheus -n monitoring --timeout=120s
kubectl wait --for=condition=ready pod -l app=grafana -n monitoring --timeout=120s
```

Open Grafana at `http://$(minikube ip):30030` — both datasources should be green and all 5 dashboards visible under the Uber folder.

Observability Deliverables

- `logback-spring.xml` in all 5 services with Loki4J appender (§11.1)
- `management.endpoints.web.exposure.include: prometheus,health,info` in all 5 services
- 5 Grafana dashboard JSON files (`user-dashboard.json`, `driver-dashboard.json`, `ride-dashboard.json`, `location-dashboard.json`, `payment-dashboard.json`) — ≥3 LogQL + ≥3 PromQL panels each, committed under `k8s/monitoring/grafana/dashboards/`
- `k8s/namespaces/monitoring-namespace.yaml` declaring the `monitoring` namespace
- `k8s/monitoring/loki/` — ConfigMap + PVC + StatefulSet + Service
- `k8s/monitoring/prometheus/` — ConfigMap (with the 5-job scrape config) + PVC + Deployment + Service
- `k8s/monitoring/grafana/` — datasources ConfigMap + dashboards ConfigMap + PVC + Deployment + NodePort Service (30030)
- Verified end-to-end: trigger an HTTP request via the gateway → log line appears in Loki within ~5s; metric counter increments in Prometheus within ~15s; both render in the corresponding service dashboard

Section 12 — Project Folder Structure

This is the canonical layout your team's repo must end up in by the end of M3. Every file path referenced elsewhere in this spec maps onto this tree.

```
uber-m3/                                # git repo root
├── pom.xml                             # parent POM — 7 modules
├── README.md
├── docker-compose.yml                  # local dev compose
├── contracts/                          # Day-0 kickoff module
│   ├── pom.xml
│   └── src/main/java/com/<teamID>/<domain>/contracts/
├── user-service/                       # S1
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/
├── driver-service/                     # S2
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/main/java/com/<teamID>/<domain>/driver/
├── ride-service/                       # S3 (saga state machine lives here)
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/main/java/com/<teamID>/<domain>/ride/
├── location-service/                   # S4
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/main/java/com/<teamID>/<domain>/location/
├── payment-service/                    # S5
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/main/java/com/<teamID>/<domain>/payment/
├── api-gateway/                        # 6th Maven module
│   ├── pom.xml                         # spring-cloud-starter-gateway-server-webflux
│   ├── Dockerfile
│   └── src/main/
├── k8s/                                # all Kubernetes manifests
│   ├── namespaces/
│   ├── secrets/
│   ├── configmaps/
│   ├── pvcs/
│   ├── statefulsets/
│   ├── deployments/
│   ├── services/
│   ├── api-gateway/
│   └── monitoring/                     # everything in `monitoring` namespace
└── .github/workflows/                  # bonus — CI/CD
    └── ci.yml
```

12.1 How Services Reference Files From Other Modules

The `contracts/` module is the mechanism that lets all 5 services share Feign interfaces, DTOs, and event records without duplicating any Java code. It is a plain Maven JAR (no Spring Boot parent, no executable) that the 5 services and the gateway depend on.

On the `com.<teamID>.<domain>.*` package convention: the package examples in this section use the placeholders `<teamID>` (your team's identifier — same one you've used since M1) and `<domain>` (the theme name, e.g., `uber`). Substitute both literally before committing — e.g., team `ABC123` working on Uber writes `com.ABC123.uber.user.service`. The team-based prefix matches the M1/M2 grader's package convention and prevents Maven groupId collisions across teams; the per-team disambiguation is also what the plagiarism-detection pipeline relies on. **Do not ship a literal `com.uber.*` or `com.<teamID>.<domain>.*` (with the placeholders unsubstituted)** — both fail the grader's compile step.

Parent pom.xml — Module Aggregator

The root `pom.xml` lists every module in build order. Maven's reactor builds `contracts` first because the 5 services declare it as a `<dependency>`:

```
    <modules>
        <module>contracts</module>
        <module>user-service</module>
        <module>driver-service</module>
        <module>ride-service</module>
        <module>location-service</module>
        <module>payment-service</module>
        <module>api-gateway</module>
    </modules>
```

contracts/pom.xml — The Shared Types Module

```
    <project>
        <modelVersion>4.0.0</modelVersion>

        <parent>
            <groupId>com.<teamID>.<domain></groupId>
            <artifactId>uber-m3</artifactId>
            <version>1.0.0</version>
        </parent>

        <artifactId>contracts</artifactId>
        <packaging>jar</packaging>

        <dependencies>
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-starter-openfeign</artifactId>
            </dependency>
        </dependencies>
    </project>
```

The `spring-cloud-starter-openfeign` dependency is required because the `@FeignClient` annotation lives on interfaces inside this module. Event records and DTOs are plain Java records and need no extra dependencies.

Each Service pom.xml — Depends on contracts

Add this to each of `user-service/pom.xml`, `driver-service/pom.xml`, `ride-service/pom.xml`, `location-service/pom.xml`, and `payment-service/pom.xml`:

```
    <dependency>
        <groupId>com.<teamID>.<domain></groupId>
        <artifactId>contracts</artifactId>
        <version>1.0.0</version>
    </dependency>
```

The api-gateway does **not** depend on `contracts` (it does not call Feign clients itself; it just forwards HTTP requests).

How Java Code Imports Across Modules

Once a service depends on `contracts`, every type defined there is importable like any other Java package. For example, user-service calling ride-service via Feign:

```java
package com.<teamID>.<domain>.user.service;

import com.<teamID>.<domain>.contracts.feign.RideServiceClient;          // from contracts module
import com.<teamID>.<domain>.contracts.dto.RideSummaryDTO;               // from contracts module
import com.<teamID>.<domain>.user.entity.User;                            // local to user-service
import com.<teamID>.<domain>.user.repository.UserRepository;              // local to user-service

@Service
public class UserRideSummaryService {
    private final UserRepository userRepository;
    private final RideServiceClient rideClient;              // Feign interface from contracts

    public UserRideSummaryService(UserRepository userRepository, RideServiceClient rideClient) {
        this.userRepository = userRepository;
        this.rideClient = rideClient;
    }

    public UserRideSummaryDTO buildSummary(Long userId) {
        User user = userRepository.findById(userId).orElseThrow();
        RideSummaryDTO summary = rideClient.getUserRideSummary(userId);
        return new UserRideSummaryDTO(user, summary);
    }
}
```

Same pattern for events — payment-service consuming `ride.completed`:

```java
package com.<teamID>.<domain>.payment.messaging.consumers;

import com.<teamID>.<domain>.contracts.events.RideCompletedEvent;        // from contracts
import com.<teamID>.<domain>.contracts.feign.UserServiceClient;          // from contracts
import com.<teamID>.<domain>.payment.entity.Payment;                      // local

@Component
public class RideEventConsumer {
    private final UserServiceClient userClient;
    private final PaymentService paymentService;

    @RabbitListener(queues = "payment.saga-listener")
    public void onRideCompleted(RideCompletedEvent event) {
        UserDTO user = userClient.getUser(event.userId());
        paymentService.createPendingPayment(event.rideId(), event.userId(), event.fare());
    }
}
```

Build Order — Maven Reactor Handles It Automatically

Run `mvn clean install` from the repo root. Maven's reactor:

1. Detects that user-service (and the other 4 services) depend on `contracts:1.0.0`.
2. Builds `contracts` first, installs it into the local Maven repo (`~/.m2/repository/com/<teamID>/<domain>/contracts/1.0.0/`).
3. Builds the 5 services in any order (no inter-service dependencies — they all only depend on `contracts`).
4. Builds `api-gateway` last (or in parallel with services — it depends on neither `contracts` nor any service).

For local Docker dev (`docker-compose up`), each service's Dockerfile copies its own JAR — the `contracts` JAR is already baked into the service JAR via Maven's shade/repackage plugin during step 3.

Why This Eliminates Cross-Slice Compile Blockers

When student A starts work on **S1-READ-DB** (which calls `RideServiceClient.getUserRideSummary(...)`), they need that interface to exist in `contracts/` so their code compiles. Day-0 kickoff (§13.2 Parallelism Strategy) ensures:

1. All 5 Feign client interfaces + all DTOs + all event records are committed to `contracts/` on Day 0, **before** any of the 15 slices begin work.
2. Student A's user-service compiles immediately because `RideServiceClient` exists in the imported `contracts` JAR — even though student G (owner of S3-READ-DB) hasn't yet implemented the matching `GET /api/rides/user/{userId}/summary` endpoint inside ride-service.

3. **Runtime testing:** student A uses `@MockBean RideServiceClient` until student G's branch merges.

12.2 Module-to-Slice Map

The 15 deliverable slices (§13.2) map onto the folder tree as follows. Use this as a per-slice checklist of which files a member touches:

Slice	Touches
S1-READ-DB	`user-service/src/main/java/.../entity/` + `controller/` + `service/` + `application.yml` (datasource block); `contracts/.../feign/RideServiceClient.java` + `PaymentServiceClient.java`; `k8s/secrets/user-postgres-secret.yaml` + `k8s/pvcs/user-postgres-pvc.yaml` + `k8s/statefulsets/user-postgres-statefulset.yaml` + `k8s/services/user-postgres-svc.yaml`; `user-service/src/main/resources/logback-spring.xml`; LogQL panels in `k8s/monitoring/grafana/dashboards/user-dashboard.json`.
S1-EVENTS	`user-service/src/main/java/.../config/UserEventConfig.java` + `messaging/publishers/UserEventPublisher.java` + `messaging/consumers/RideEventConsumer.java`; `contracts/.../events/UserRegisteredEvent.java` + `UserDeactivatedEvent.java`; `k8s/configmaps/user-service-configmap.yaml` + `k8s/deployments/user-service-deployment.yaml` + `k8s/services/user-service-svc.yaml`; PromQL panels in `user-dashboard.json`.
S1-INFRA	user-service route block in `api-gateway/src/main/resources/application.yml`; user-service scrape job in `k8s/monitoring/prometheus/prometheus-configmap.yaml`; final assembly of `user-dashboard.json`. **Shared infra:** entire `api-gateway/` Maven module (incl. `JwtGatewayFilter.java`) + `k8s/api-gateway/gateway-deployment.yaml` + `gateway-service.yaml` (NodePort 30080) + `k8s/statefulsets/mongo-statefulset.yaml` + `k8s/services/mongo-svc.yaml` + `k8s/pvcs/mongo-pvc.yaml`.
S2-READ-DB	driver-service equivalent of S1-READ-DB, plus implementation of `GET /api/drivers/{id}/availability`.
S2-EVENTS	driver-service equivalent of S1-EVENTS — `DriverEventConfig` + publishers (status-changed, rated, document.verified) + consumers (ride.placed, ride.completed, ride.cancelled); `contracts/.../events/DriverStatusChangedEvent.java` + `DriverRatedEvent.java` + `DriverDocumentVerifiedEvent.java`.
S2-INFRA	driver-service route + scrape entry + dashboard. **Shared infra:** `k8s/namespaces/monitoring-namespace.yaml` + entire `k8s/monitoring/loki/` + `k8s/statefulsets/redis-statefulset.yaml` + redis Service + PVC.
S3-READ-DB	ride-service entity (incl. saga statuses on Ride) + new endpoints `GET /api/rides/user/{id}/{summary,active-count,completed-count}`, `GET /api/rides/driver/{id}/{summary,active-count,completed-count}`; `contracts/.../feign/DriverServiceClient.java` + `UserServiceClient.java` + `LocationServiceClient.java`; ride-postgres K8s; logback + LogQL panels.
S3-EVENTS	ride-service `saga/` package (S3-F4 complete, S3-F7 cancel, payment-event consumers) + `RideEventConfig` + publishers (ride.placed/completed/cancelled); `contracts/.../events/RideCompletedEvent.java` etc.; ride-service Deployment + Service + ConfigMap; PromQL panels.
S3-INFRA	ride-service route + scrape entry + dashboard. **Shared infra:** entire `k8s/monitoring/prometheus/` (Deployment + ConfigMap holding the full 5-job scrape config + PVC + Service) +

	`k8s/statefulsets/neo4j-statefulset.yaml` + neo4j Service + PVC.
S4-READ-DB	location-service entity + new endpoint `GET /api/locations/driver/{driverId}/recent` (saga pre-check); `contracts/.../feign/DriverServiceClient.java` (read uses); location-postgres K8s; logback + LogQL panels.
S4-EVENTS	location-service `LocationEventConfig` + publishers (location.tracked — optional/audit) + consumers (ride.placed/completed/cancelled); `contracts/.../events/LocationTrackedEvent.java`; location-service Deployment + Service + ConfigMap; PromQL panels.
S4-INFRA	location-service route + scrape entry + dashboard. **Shared infra:** entire `k8s/monitoring/grafana/` (Deployment + datasources ConfigMap + dashboards ConfigMap embedding all 5 JSONs + PVC + NodePort 30030) + `k8s/statefulsets/cassandra-statefulset.yaml` + cassandra Service + PVC.
S5-READ-DB	payment-service entity + new endpoint `GET /api/payments/user/{userId}/total`; `contracts/.../feign/UserServiceClient.java` + `RideServiceClient.java` + `DriverServiceClient.java` (read uses); payment-postgres K8s; logback + LogQL panels.
S5-EVENTS	payment-service `PaymentEventConfig` + publishers (payment.initiated/completed/failed/refunded) + consumers (ride.completed → create PENDING payment, ride.cancelled → refund via S5-F12 Strategy); `contracts/.../events/Payment*.java`; payment-service Deployment + Service + ConfigMap; PromQL panels.
S5-INFRA	payment-service route + scrape entry + dashboard. **Shared infra:** `k8s/statefulsets/rabbitmq-statefulset.yaml` + rabbitmq Service (5672 + 15672) + PVC + `k8s/statefulsets/elasticsearch-statefulset.yaml` + ES Service + PVC + saga end-to-end test scenarios A/B/C from §8.6 (JUnit integration tests).

Section 13 — Work Distribution

13.1 Branch Format

`feat/M3/<scope>/<ID>/<studentID>`

Commit format: `feat(<scope>): <description> (studentID)`

13.2 The 15 Deliverables

Rule: Each deliverable is a vertical slice that touches all parts of M3 — Java code, Kubernetes manifests, and observability artifacts. No deliverable is purely Java, K8s, or YAML. The 15 slices are designed so every team member works in parallel without blocking anyone else (see “Parallelism Strategy” below the table).

The 15 deliverables are organized as **5 services × 3 vertical slices per service**:

- **Slice A — Read & DB:** DB isolation, outbound Feign clients, exposed read endpoints, Postgres K8s, ≥3 LogQL panels, Logback config.
- **Slice B — Events & Saga:** RabbitMQ topology, publishers/consumers, saga participation, Spring Boot K8s, ≥3 PromQL panels, actuator config.
- **Slice C — Cross-Cutting Infra:** that service’s gateway route entry + scrape job entry + dashboard JSON aggregation, plus one assigned shared-infra item.

#	Branch ID	Service	Work (Java + K8s + Observability)
1	S1-READ-DB	user	DB isolation (datasource → uberdb-users, any cross-service `@ManyToOne` → `Long`); `RideServiceClient` + `PaymentServiceClient` Feign interfaces with try-catch error handling + correlation interceptor; user-postgres K8s (StatefulSet + PVC + Secret + headless Service); `logback-spring.xml`; ≥3 LogQL panels for user-service dashboard.
2	S1-EVENTS	user	`user.events` TopicExchange + publishers (user.registered, user.deactivated); consumer queue `user.ride.saga-listener` + DLQ; consumers for `ride.completed` / `ride.cancelled` updating user stats; user-service K8s Deployment + ClusterIP Service + ConfigMap; actuator config; ≥3 PromQL panels for user-service dashboard; S1-F3, S1-F4, S1-F6, S1-F9 Java refactor (Feign + event publish).
3	S1-INFRA	user	user-service gateway route entry; user-service scrape job entry in `prometheus.yml`; final user-service dashboard JSON file. **Shared infra owned by this slice:** api-gateway Maven module (6th module in root pom.xml) + `JwtGatewayFilter` (Java) with `X-User-Id` / `X-User-Role` / `X-Correlation-ID` forwarding + `/api/auth/**` bypass + gateway K8s Deployment + NodePort Service (30080) + Mongo K8s StatefulSet + Service.
4	S2-READ-DB	driver	DB isolation (datasource → uberdb-drivers); `GET /api/drivers/{id}/availability` exposed; `RideServiceClient` + `UserServiceClient` Feign interfaces with error handling; driver-postgres K8s (StatefulSet + PVC + Secret + Service); `logback-spring.xml`; ≥3 LogQL panels for driver-service dashboard.
5	S2-EVENTS	driver	`driver.events` TopicExchange + publishers (driver.status-changed, driver.rated, driver.document.verified); consumer queue `driver.ride.saga-listener` + DLQ; consumers for `ride.placed` / `ride.completed` / `ride.cancelled`; driver-service K8s Deployment + Service + ConfigMap; actuator; ≥3 PromQL panels; S2-F3, S2-F4, S2-F7, S2-F8, S2-F12 Java refactor.
6	S2-INFRA	driver	driver-service gateway route + scrape job entry + final dashboard JSON. **Shared infra owned by this slice:**

			monitoring namespace YAML + Loki K8s (StatefulSet + ConfigMap + PVC + Service named `loki`) + Redis K8s StatefulSet + Service.
7	S3-READ-DB	ride	DB isolation (datasource → uberdb-rides); add saga statuses to Ride enum; expose `GET /api/rides/user/{userId}/summary`, `active-count`, `completed-count`, `GET /api/rides/driver/{driverId}/summary`, `active-count`, `completed-count`; `DriverServiceClient` + `UserServiceClient` + `LocationServiceClient` Feign interfaces with error handling; ride-postgres K8s (StatefulSet + PVC + Secret + Service); `logback-spring.xml`; ≥3 LogQL panels for ride-service dashboard.
8	S3-EVENTS	ride	`ride.events` TopicExchange + publishers (ride.placed, ride.completed, ride.cancelled); consumer queue `ride.saga-feedback` + DLQ; consumers for `payment.initiated` / `payment.completed` / `payment.failed` (compensation trigger)/ `payment.refunded`; ride-service K8s Deployment + Service + ConfigMap; actuator; ≥3 PromQL panels; S3-F2, S3-F4 (saga trigger), S3-F7 (cancel), S3-F11, S3-F12 Java refactor.
9	S3-INFRA	ride	ride-service gateway route + scrape job entry + final dashboard JSON. **Shared infra owned by this slice:** Prometheus K8s (Deployment + ConfigMap holding the full 5-job `prometheus.yml` + PVC + Service named `prometheus`) + Neo4j K8s StatefulSet + Service.
10	S4-READ-DB	location	DB isolation (datasource → uberdb-locations); expose `GET /api/locations/driver/{driverId}/recent` (new saga pre-check endpoint); `DriverServiceClient` Feign interface with error handling; location-postgres K8s (StatefulSet + PVC + Secret + Service); `logback-spring.xml`; ≥3 LogQL panels for location-service dashboard.
11	S4-EVENTS	location	`location.events` TopicExchange + optional publisher (location.tracked); consumer queue `location.ride.saga-listener` + DLQ; consumers for `ride.placed` / `ride.completed` (mark final ping with rideId)/ `ride.cancelled`; location-service K8s Deployment + Service + ConfigMap; actuator; ≥3 PromQL panels; S4-F3, S4-F9 Java refactor.
12	S4-INFRA	location	location-service gateway route + scrape job entry + final dashboard JSON. **Shared infra owned by this slice:** Grafana K8s (Deployment + datasources ConfigMap pointing at Loki & Prometheus + dashboards ConfigMap embedding all 5 service dashboards + PVC + NodePort Service on 30030) + Cassandra K8s StatefulSet + Service.
13	S5-READ-DB	payment	DB isolation (datasource → uberdb-payments); expose `GET /api/payments/user/{userId}/total?startDate=&endDate=`; `UserServiceClient` + `RideServiceClient` + `DriverServiceClient` Feign interfaces with error handling; payment-postgres K8s (StatefulSet + PVC + Secret + Service); `logback-spring.xml`; ≥3 LogQL panels for payment-service dashboard.
14	S5-EVENTS	payment	`payment.events` TopicExchange + publishers (payment.initiated, payment.completed, payment.failed, payment.refunded); consumer queue `payment.saga-listener` + DLQ; consumers for `ride.completed` (create PENDING payment → publish `payment.initiated`) and `ride.cancelled` (refund via S5-F12 Strategy → publish `payment.refunded`); payment-service K8s Deployment + Service + ConfigMap; actuator; ≥3 PromQL panels; S5-F3, S5-F4, S5-F10 Java refactor.
15	S5-INFRA	payment	payment-service gateway route + scrape job entry + final dashboard JSON. **Shared infra owned by this slice:** RabbitMQ K8s (StatefulSet + Service exposing 5672 + 15672) + Elasticsearch K8s StatefulSet + Service + saga end-to-end

			test scenarios A/B/C from §8.6 implemented as JUnit integration tests.

Parallelism Strategy — How All 15 Members Work Without Blocking Each Other

The 15 slices are designed so nobody waits for anyone else. The key is **contract-first development**: every cross-service interface is agreed in a kickoff meeting on Day 1, written down, and committed before any feature work starts. From that moment, each member writes against the contract — not against another member's implementation — so they can compile, test (with mocks), and deploy their slice independently.

Day-0 kickoff contracts (committed by the team lead, ~2 hours):

1. **Feign client interfaces** — every `@FeignClient` interface signature (e.g., `RideServiceClient.getUserRideSummary`) and the DTOs they return (`RideSummaryDTO`, `DriverRideSummaryDTO`, etc.). Committed once to a `contracts/` Maven module that all services depend on.
2. **Event payload records** — every `record` class (`RideCompletedEvent`, `PaymentFailedEvent`, ...) is added to that same `contracts/` module. Routing keys + exchange names are fixed in §2.9 (no team debate).
3. **New endpoint paths + DTO shapes** — exact path, query params, response JSON. Already documented in each service's "New Endpoints" table (§3–§7).
4. **K8s Service names** — `loki`, `prometheus`, `rabbitmq`, `<svc>-postgres` — fixed up-front so DNS resolves correctly across slices.
5. **Shared YAML stub files** — `api-gateway/application.yml` (with route placeholders), `prometheus-configmap.yaml` (with scrape-job placeholders), `grafana-dashboards.yaml` ConfigMap (referencing 5 dashboard JSON paths). Each "INFRA" slice owns the creation of one stub; each service slice fills in its own block.

Compile-time independence — once the `contracts/` module is in place, slice 1's `RideServiceClient.getUserRideSummary(...)` call compiles even if slice 7 hasn't implemented `GET /api/rides/user/{userId}/summary` yet. The interface is the only thing slice 1 needs to compile and unit-test.

Runtime independence (mocking) — for local dev each slice uses `@MockBean` on Feign clients and Testcontainers RabbitMQ. A slice can run, deploy, and verify in isolation without the other 14 slices being merged.

Disjoint file ownership — each slice writes to its own packages and YAML blocks. The only shared YAML files are `api-gateway/application.yml`, `prometheus.yml`, and `grafana-dashboards.yaml` — these have a stable structure agreed at kickoff so each slice edits only its assigned block. Merge conflicts are minimized to non-existent.

Deploy-time independence — when a slice's branch is ready, it merges into main whenever; the merge order is not prescribed because no slice depends on another slice being merged first. Integration verification (saga end-to-end, gateway routing) happens after all 15 are merged, owned by S5-INFRA.

13.3 Team Size Mapping

Team size	Mapping
15 members	1 deliverable per member, exactly. Default mapping.
14 members	One member takes 2 deliverables. Recommended pairing: `S<i>-READ-DB` + `S<i>-INFRA` for any service whose INFRA slice is light (e.g., S2-INFRA if Loki is the team's most-familiar tool).
13 members	Two members each take 2 deliverables. Recommended: pair `S<i>-READ-DB` + `S<i>-INFRA` for two services whose INFRA assignments are smaller (e.g., S2 + S4).

13.4 Merge Order

Because the contract-first design eliminates compile-time dependencies, **merge order is unconstrained** — branches can be merged in any order, as long as the `contracts/` module exists in main first.

1. **Day 0:** Team lead merges the `contracts/` Maven module + the 3 stub YAML files (`api-gateway/application.yml`, `prometheus-configmap.yaml`, `grafana-dashboards.yaml`) into main.
2. **Day 1 onwards:** All 15 slices proceed in parallel; each merges to main when ready. No slice blocks another.
3. **Final integration:** Once all 15 slices are merged, S5-INFRA owner runs the saga end-to-end test scenarios A/B/C (§8.6) and signs off.

Section 14 — Evaluation Format

14.1 Individual Presentation (~5 minutes per member)

Each member presents the branch they implemented and you will need to answer questions about your part of work.

14.2 Demo Requirements

The team (like one member at least) must be able to run the full project from the cluster:

```
kubectl get pods -n uber                          # all pods Running
kubectl logs <your-service-pod> -n uber           # your service logs
curl http://$(minikube ip):30080/api/<endpoint> # your feature end-to-end
```

For saga branch owners: demonstrate the Ride Lifecycle Saga by triggering `PUT /api/rides/{id}/complete` and showing the event ripple in driver-service, location-service, and payment-service logs.

Section 15 — Bonus

Bonus	Description
Full Testing Suite	(1) Unit tests for service business logic with `@MockBean` on all Feign clients. (2) RabbitMQ consumer integration tests with Testcontainers — publish an event, assert the consumer processes it and mutates the local DB. (3) Saga E2E test: trigger S3-F4, assert `payment.initiated` is received; then inject payment failure, assert compensation runs.
CI/CD Pipeline	GitHub Actions: on push to `feat/*` → Maven build + JUnit + Docker build. On push to `main` → push images to a container registry. Submit as `.github/workflows/ci.yml`.
Circuit Breaker	Add `spring-cloud-starter-circuitbreaker-resilience4j` to services with Feign calls. Configure fallback responses. Demonstrate: circuit opens on repeated failures, fallback activates, circuit recovers.
Ingress	Replace NodePort on api-gateway with an Ingress resource. `minikube addons enable ingress`, configure Ingress with path-based routing to the gateway.
Horizontal Pod Autoscaler	HPA on ride-service (highest traffic). CPU threshold ≥ 50%. Requires `metrics-server` in MiniKube. Demonstrate scale-out under simulated load.

Section 16 — Critical Rules

1. **No cross-service JDBC.** After M3, no service opens a JDBC connection to another service's database. Zero tolerance.
2. **Feign for reads. RabbitMQ for side-effects.** Use Feign when you need data to continue processing. Use RabbitMQ when triggering a state change in another service.
3. **Auto ACK with DLQ routing.** Use Spring's default `acknowledge-mode: auto` with `default-requeue-rejected: false`. Spring ACKs the message when the listener method returns normally and rejects when it throws — after retries are exhausted, rejected messages flow to the DLQ via the queue's `x-dead-letter-exchange` argument (no manual `basicAck` / `basicNack` calls).
4. **DLQ for every queue.** Every consumer queue has a dead-letter queue. Failed messages are never silently dropped.
5. **PostgreSQL 17.** Not PG18 — breaks Hibernate native query implicit cast operator resolution.
6. **StatefulSet for all databases.** Never use plain `Deployment` for a stateful database.
7. **Explicit constructor injection.** Consistent with M1/M2 — no Lombok.
8. **JWT validation at gateway.** Individual services retain their M2 JWT filter for defense-in-depth, but the gateway is the public-facing validator.
9. **No new tests added during grading.** Like M1/M2 the grader-provided test suite is the source of truth.
10. **15 deliverables, contract-first parallel work.** No slice waits for another slice's implementation; the `contracts/` module is the only Day-0 dependency.
11. **Consumers must be idempotent.** RabbitMQ delivery is at-least-once: with `acknowledge-mode: auto` + `default-requeue-rejected: false` + `max-attempts: 3` (Rule 3), a transient listener failure causes Spring to retry the same message before DLQ routing. Consumers therefore see duplicates. Use **state-based idempotency** — check the target row's status before mutating (e.g., a `ride.completed` consumer that increments driver earnings should first read the driver row and skip if the rideId has already been counted; a `payment.completed` consumer for ride-service should check `ride.status != PAID` before transitioning). State-based idempotency is sufficient for M3 scope; explicit idempotency keys are not required.